

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2017.DOI

A survey on computation offloading and current trends

ANANT KUMAR¹, AMRIT PAL(Member, IEEE)

¹School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, Tamilnadu 600127, India

Corresponding author: Amrit Pal (amrit.pal@vit.ac.in).

This work was supported by Vellore Institute of Technology, Chennai

ABSTRACT Computation offloading has proven to be an important technique in modern computing to improve performance by saving energy consumption and optimizing resource utilization by transferring computational tasks from devices constrained by resources, such as mobile devices, Internet of Things (IoT), nodes, and edge systems, to more powerful remote servers, cloud platforms, or edge computing nodes. This survey offers a critical view of computation offloading from architectural models to decision-making strategies in real-world applications. Offloading strategies may differ between full and partial offloading, highlighting significant key challenges such as network latency, task scheduling, security risk, and resource allocation. Additionally, this section provides an overview of various machine learning techniques, including supervised machine learning, unsupervised machine learning, deep learning, and reinforcement learning, which are utilized for real-time adaptive and secure task distribution. Furthermore, advancements in 5G networks and edge intelligence have significantly reduced computational delay, making offloading more efficient for applications that demand low latency, such as cloud gaming, autonomous vehicles, and unmanned aerial vehicles used for surveillance purposes. The survey enhances computation offloading strategies for future distributed computing environments by reviewing state-of-the-art techniques and methodologies. A part of this analysis is focused on the simulation model analyses used in various computation offloading schemes. Ultimately, we believe that there is still significant scope for enhancing computation offloading, optimizing energy consumption, and reducing latency to uphold the overall quality required. This survey paper explores the potential for future research directions in this area.

INDEX TERMS Computation Offloading, Machine learning techniques, Mobile edge computing, Mobile cloud computing, Resource allocation, Communication network, Energy efficient offloading.

I. INTRODUCTION

THE fast evolution of mobile apps today creates a high demand for processing capability and power efficiency in user equipment [1] [2]. Most applications are very compute-intensive and sometimes require much computing power that usually exceeds what can be offered by UEs [3]; these are sensitive to latency as well. Because of this, running certain applications locally can result in ineffective processing, increased energy consumption, and even task failure [4]. Computation offloading has become a feasible solution to these problems, allowing computational tasks to be transferred from devices with limited resources to more powerful remote processing nodes (RPNs), such as cloud servers, clusters, edge computing frameworks, or fog computing infrastructure [5]. The potential of computing offloading to improve application performance and system

efficiency [6] has attracted a lot of research attention. Several industries, including robotics [7] [8], cloud gaming [9] [10], unmanned aerial vehicles (UAVs) [11] [12] [13], and autonomous vehicles [14] [15], have seen substantial research on offloading. Many applications emerged from the adaptability and flexibility of offloading systems, highlighting their significance in current computing parameters. Computation offloading has also gone through significant evolution over the past decade, as shown in Figure 1. In the early days of cloud offloading between 2010 and 2013, mobile cloud computing gained popularity; however, high latency and cloud connectivity limitations limited its use for delay-sensitive applications. In the years 2014 to 2016, edge and fog computing came to the forefront, with MEC mitigating latency issues by serving data closer to the users. Between 2017 and 2019, the trend of intelligent offloading began with the use of reinforce-

ment learning (RL) and Q-learning for dynamic and partial task offloading that allowed swift adaptive decision-making. The years 2020 to 2022 marked the increase in federated and privacy-preserving edge intelligence, merging federated learning with secure aggregation and AI-driven task scheduling to safeguard data and optimize edge resources. From the year 2023, the collaboration and green edge computing marked the shift in attention, focusing on energy-efficient offloading, edge-cloud orchestration, and meeting sustainability targets. Mobile cloud computing (MCC) was one of the first methods for offloading computations [16]. In MCC [17], tasks were sent to remote cloud servers for processing. As shown in Figure 5, the computational offloading system model comprises cloud, mobile edge computing (MEC) [18], and end-user devices. An MEC server is located at a base station and processes tasks; thus, nearby users further reduce the latency and optimize resources. Devices continuously make local, offload to MEC, or cloud resources decisions depending on improved efficiency. However, the fundamental latency of cloud computing and overcrowded networks enabled the development of more regional possibilities. Multi-access Edge Computing (MEC) and Fog Computing (FC) were developed to address these issues by processing capacity closer to the end users. These systems reduced the delay in the transmission and improved the real-time processing capabilities, making them suitable for delay-sensitive applications [19]. Though computation offloading offers its advantages, the disadvantages are paramount, especially in dynamic environments [20]. One of the main issues is the dynamic rate of task arrival, especially in applications such as autonomous driving, where unexpected workloads may lead to resource overload or under-utilization [4]. Also, a major worry is network overload created by sending massive amounts of duplicate data, especially in robotics systems, UAV networks, and cloud gaming [5]. An additional Major challenge is resource allocation optimization, which entails efficient use of computational and communicational and communication resources [1]. Performance improvement depends on signal processing, adaptive task scheduling, and effective bandwidth usage [6]. To enhance energy economy and computing effectiveness, researchers have considered several strategies, such as task splitting and multi-level offloading concepts [16]. The nature of computationally expensive operations is highly associated with offloading strategy design, as well as efficiency. Classifying the computation tasks as being either static or dynamic would require the understanding and characterization of generating data features [19]; static ones rely on known inputs and pre-configured sizes, which are simpler, while the variability in the dynamic cases prevents the advanced optimization of these due to uncertain varying data sizes or arrival rates [20]. Such advanced techniques as adaptive resource allocation and redundancy removal have been proposed for the purpose of converting static tasks into dynamic ones and improving overall system efficiency. While Previous surveys [21] [22] [23] [24] [25] [26] [5] [2] [27] primarily focused on specific offloading strategies, such as reinforcement learning

or AI-based methods, they often neglected the integration of different learning paradigms. Therefore, this papers bridge the gap by offering a unified framework that incorporates supervised learning, unsupervised learning, deep learning, reinforcement learning, federated learning, and blockchain-based approaches, making it one of the most holistic surveys in this field to date. It uniquely differentiates between static and dynamic tasks, providing mathematical models for Quality of Service (QoS) parameters and the decision-making process. Moreover, the inclusion of publication trends analysis (2018-2025), keywords statistics, and system modeling across MEC, cloud, and fog environments adds valuable insight into current research directions. In addition to our paper, we also discuss real-time constraints, task scheduling, and anomaly detection, which are important topics that have not been fully explored in previous research. Therefore, our survey serves as a comprehensive and future-oriented resource for researchers and developers involved in mobile and edge computing environments.

The rest of this paper is organized as follows. 2. Global Contributions and Trends in computation offloading research, 3. Computation offloading, 4. Task Generation, 5. Communication and computation, 6. Resource allocation and decision making, 7. An overview of computation offloading, 8. Problem architecture: system model, 9. Computation offloading datasets, 10. Tool-based analysis in computation offloading, 11. Conclusion and future work. 12. References.

II. GLOBAL CONTRIBUTIONS AND TRENDS IN COMPUTATION OFFLOADING RESEARCH

Computation offloading research has gained interest surrounding developments in cloud computing and edge computing. Researchers worldwide are focusing on optimizing resource allocation, reducing latency, and improving energy efficiency.

A. RESEARCH METHODOLOGY AND REVIEW PROTOCOLS

The review focused on publications from 2016 to early 2025 (till June). For data sources and research approaches, some of the widely recognized digital libraries were used to conduct an exhaustive literature survey: IEEE Xplore, ACM Digital Library, ScienceDirect, Elsevier, and SpringerLink. These databases were chosen for their sweeping coverage of peer-reviewed and top-cited publications in mobile computing, edge/cloud architecture, and machine learning based systems. Articles that include the following keywords: "Computation offloading", "Mobile edge computing", "Cloud offloading", "Task scheduling and resource allocation", "Machine learning in offloading", "Energy-aware offloading", and "AI-driven Architecture".

B. THE MOST PAPER PUBLISHED DURING 2016-2025

The computation offloading-based publications listed in Google Scholar publications in the increasing domain of interest certainly represent the trend increase between 2016

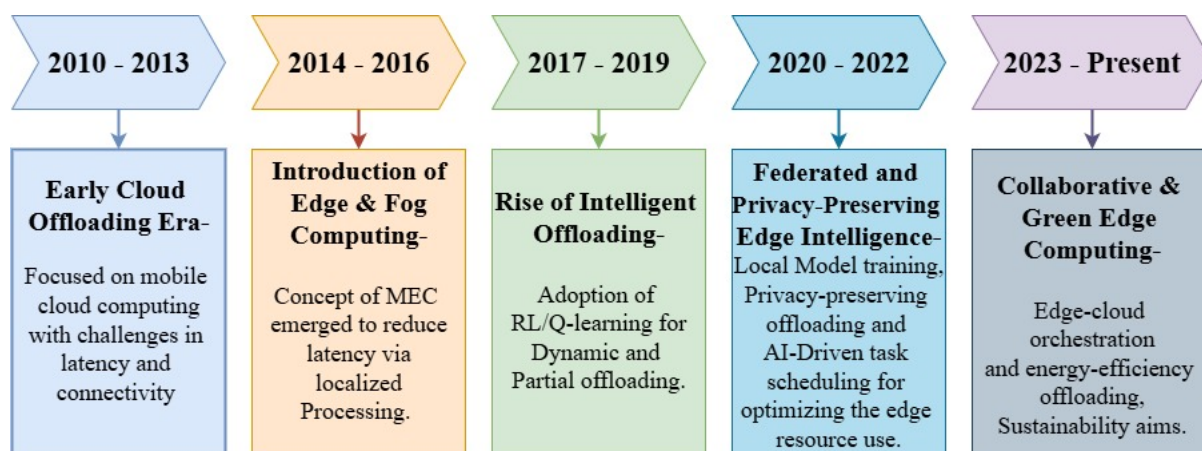


FIGURE 1. Year-Wise Evolution of Computation Offloading Edge Computing.

and 2025 (till June), as shown by this increasing interest in Figure 2. The chart displays a steady and consistent increase in the number of publications from 2016 to 2025 (till June), suggesting a rising academic interest and technological significance. From 2016 to 2018, there were 3,633 publications, indicating the early stage of research in this field. However, this number gradually increased, reaching 2,880 in 2019 and 4,040 in 2020. The significant expansion in the industry can be credited to the rapid advancement of mobile cloud computing, Internet of Things (IoT), and edge computing technologies during that time. The trend continued with 4,900 papers in 2021, 5,990 in 2022, and 7,060 in 2023, showing the focus on high-level topics like latency reduction, energy-efficient offloading, task scheduling algorithms, and the use of machine learning for intelligent decision-making. The topmost drawing appeared in 2024, with 8,810 publications, probably due to the integration of AI with edge computing, federated learning coming into play, and 5G networks being rolled out with efficient offloading strategies. The substantial decrease to 2220 publications in 2025 (till June) should be approached with caution, as it may be a result of incomplete data or delays in indexing rather than a true decrease in interest. Overall, the chart illustrates the growing significance of computation offloading in facilitating low-latency, resource-efficient computing for real-time and distributed applications like cloud gaming, autonomous vehicles, and smart healthcare systems.

C. THE MOST POPULAR KEYWORDS IN THE CATEGORY OF 15 ARTICLES

The most famous keywords across 15 research articles related to computation offloading are shown in Figure 3 and 4, and for categories shown in Table 1, while locating a few key areas within a domain. "Mobile Edge Computing" emerged as the most frequently used keyword, appearing in 29% of the surveyed articles, indicating a strong research emphasis on shifting computational tasks from centralized cloud environments to decentralized edge nodes. This transition addresses

challenges such as latency, bandwidth limitations, and real-time responsiveness in applications like autonomous driving and augmented reality. "Computation Offloading" itself remains a core focus, with a frequency of 20%, reflecting ongoing interest in optimizing task delegation between user devices and remote servers. The growing prevalence of artificial intelligence is reflected in the frequent use of terms like "Deep Reinforcement Learning" (12%), "Machine Learning" (5%), and "Deep Learning" (7%). These approaches are being used to create intelligent offloading strategies that can adjust to changing network conditions and user needs. Energy efficiency is a significant concern, as demonstrated by the inclusion of "Energy Efficient Offloading" (9%), emphasizing the importance of reducing energy consumption in mobile and IoT devices during the offloading process. Furthermore, keywords like "Resource Allocation" and "Communication Network" emphasize the significance of efficient resource management and dependable connectivity within offloading architectures. Overall, the keyword distribution highlights a convergence of trends towards intelligent, energy-aware, and localized computation offloading models, driven by the increasing complexity and requirements of contemporary mobile computing environments.

D. ABBREVIATIONS AND ACRONYMS

III. COMPUTATION OFFLOADING

Computation offloading refers to when computing activities are transferred from a resource-constrained device [28], such as a mobile phone or an IoT device, in the presence of a more powerful remote processing node(RPN). The RPNs may include fog compute nodes, cloud servers, edge servers, or other high-performance computing systems [29]. The goal of computation offloading is to increase processing efficiency while minimizing energy consumption and, therefore, reducing the computational burden to enhance mobile and embedded system performance. Early pioneering systems such as MAUI [30] and Clone Cloud [31] laid the foundation for computation offloading. MAUI introduced dynamic, fine-

TABLE 1. Most popular keywords in the category of 15 articles.

S.No	Keywords	Number of Usage	%	Cumulative(%)
1.	Computation Offloading	9	20	20
2.	Deep Learning	3	7	27
3.	Mobile Edge Computing	13	29	55
4.	Machine Learning	2	5	60
5.	KNN	1	2	62
6.	Mobile Cloud Computing	2	5	66
7.	Communication Network	2	5	71
8.	Energy efficient Offloading	4	9	80
9.	Deep Reinforcement Learning	5	12	91
10.	Resource Allocation	4	9	100

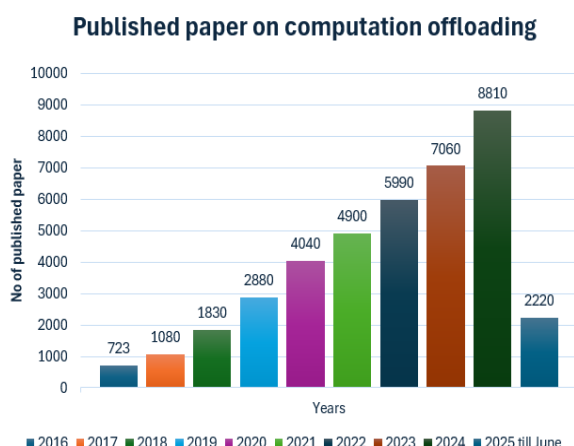


FIGURE 2. The quantity of computation offloading publications published annually between 2016 to 2025 (till June) (based on collected data from Google Scholar using the keywords "Computation Offloading").

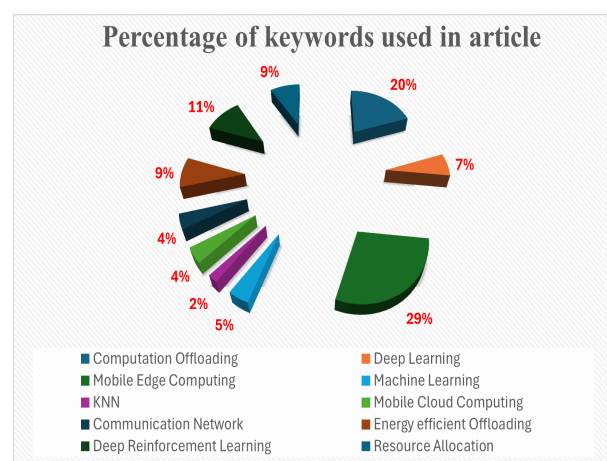


FIGURE 4. Percentage of keywords mentioned in computation offloading research.

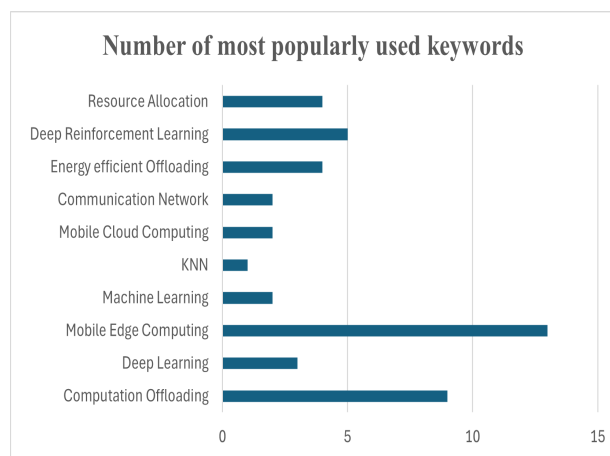


FIGURE 3. Frequency of popular keywords in computation offloading Research.

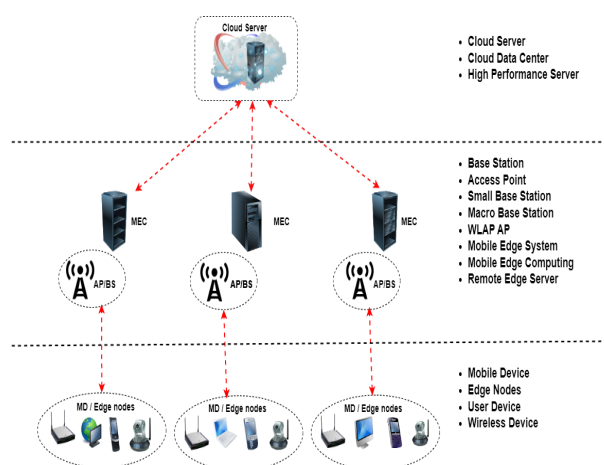


FIGURE 5. Hierarchical architecture of computation offloading in edge computing

grained code partitioning to save mobile device energy while maintaining performance, whereas CloneCloud enables elastic execution by migrating entire application threads to the cloud, reducing execution time and energy usage. The design principles from these works, dynamic task partitioning, migration transparency, and cost-benefit modeling, remain relevant and have influenced modern edge/fog computation

ABBREVIATIONS		
1	CNN	Convolutional Neural Network
2	UE	User equipment
3	RPN	Remote processing node
4	UAV	Unmanned aerial vehicle
5	MCC	Mobile cloud computing
6	MEC	Mobile edge computing
7	FC	Fog computing
8	ML	Machine learning
9	MEC	Multi-access edge computing
10	MES	Mobile edge system
11	KDN	Knowledge-Define networking
12	KNN	K-nearest neighbor
13	MCOP	Min-cost offloading partitioning
14	RPN	Remote processing node
15	BW	Bandwidth
16	RTT	Round-trip time
17	DT	Decision tree
18	RF	Random Forest
19	SVM	Support vector machine
20	ANN	Artificial neural network
21	DNN	Deep neural network
22	RNN	Recurrent neural network
23	LSTM	Long short-term memory
24	PSO	Particle Swarm Optimization
25	HSMM	Hidden Semi-Markov Model
26	HHMM	Hierarchical Hidden Markov Mode
28	AHP	Analytic hierarchy process
29	PCA	Principal component analysis
30	GPS	Genetic simulated particle swarm optimization
31	DCOA	Dynamic Computation Offloading Algorithm
32	GMM	Gaussian mixture models
33	EEO	Energy-efficient computation offloading
34	DAG	Directed Acyclic Graph
35	MILP	Mixed-integer Linear Programming
36	EPED	Efficient Deep Learning-based Offloading Algorithm
37	SARSA	State-Action-Reward-State-Action
38	WLAN	Wireless Local Area Network
39	DQN	Deep Q-network
40	FPS	Frames per second
41	DDPG	Deep deterministic policy gradient
42	DRL	Deep reinforcement learning
43	DROO	Deep Reinforcement Learning-based Online Offloading
44	MDP	Markov decision process
45	PDS	Post-Decision State
46	GAIA	Global Astrometry Interferometer for Astrophysics
47	IoT	Internet of things
48	NOMA	Non-orthogonal multiple access
49	EEDOS	Energy efficient and delay-aware offloading scheme
50	QIPSO	Quantum-Inspired Particle Swarm Optimization
51	YOLO	You Only Look Once
52	MPC	Model Predictive Control
53	TD3	Twin Delayed Deep Deterministic Policy Gradient
54	ARMS	Adaptive Resource Management System
55	DDoS	Distribution denial-of-service

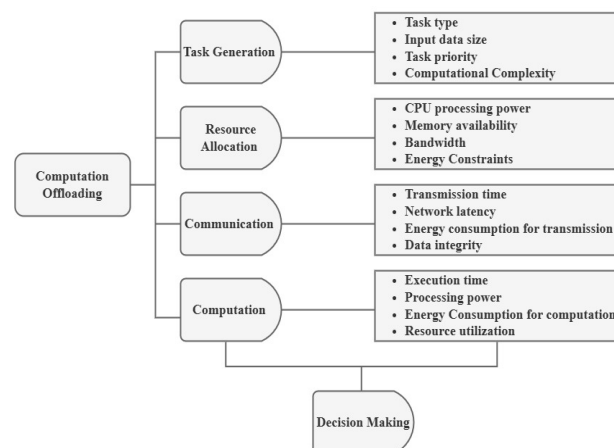


FIGURE 6. Computation offloading: A holistic decision framework.

architecture and AI-assisted offloading strategies. As shown in Figure 6, computation offloading is a process that consists of a series of steps: task generation, resource allocation, communication, and computation that further leads to decision-making.

IV. TASK GENERATION

Task generation in computation offloading refers to the process of identifying, segmenting, and preparing computational tasks that can be transferred from a local device to a more potent remote server, cloud, or edge computing system. This computation offloading task generation includes task identification and analysis, and preparation for offloading. The step includes analyzing the complexity of the task, data size, dependencies, and latency requirements. High-performance computing tasks such as video processing, ML inference, and sensor data processing are usually considered for offloading. This is where the system makes the decision whether the task should be processed locally or remotely based on some pre-determined criteria such as energy consumption, bandwidth availability, and urgency [32].

A. COMPUTATION TASKS

A basic unit of computation that consists of a request and associated data is called a task in computation offloading. When a task is generated by user equipment (UE), such as a smartphone or an Internet of Things device [2], transfer requests and data need to be processed locally or forwarded to be offloaded to a cloud server or edge device for execution. As shown in Table 2, tasks can be classified as static or dynamic. Periodically, a static task with constant data size and CPU requirement over time [33]. In contrast, a dynamic task changes in real-time circumstances [34], such as varying data size or request frequency, each with unique characteristics, challenges, and optimization techniques:

B. STATIC TASKS

Static tasks in computation offloading have well-defined characteristics and include fixed data size, a predictable amount of processing required, and deadlines for execution. Because the parameters remain fixed during execution, static tasks tend to be easier to manage than dynamic ones. A common example of a static task is the processing of pre-recorded video, where video compression or transcoding [10] follows a known computational pattern and thus maintains a constant set of processing requirements. Similarly, image recognition in stored datasets operates on fixed input sizes and predefined computational needs. Cloud computing provides another frequent instance where task assignment to servers is made based on predetermined processing times and available resources [2]. Static task offloading remains apparent in spite of challenges like reduced transmission and processing delays [35]. In addition, the distribution of the computation tasks greatly affects resource utilization: computational loads must be distributed effectively across remote processing nodes. Efficient power consumption strategies [36] must be in place while maintaining optimal performance excellence for the execution of the task. The challenges will be met in intelligent scheduling and resource management with a consequence of higher efficiency and performance for static task offloading [37].

C. DYNAMIC TASKS

Dynamic tasks vary in size, arrival rate, and computational requirements; their management becomes even more complicated compared to static tasks. These tasks are typically present in real-time applications like autonomous systems, cloud gaming, and UAV surveillance, where the demand for workload keeps shifting unexpectedly. For instance, an autonomous vehicle [14] [15] might process different sensor data depending on the prevailing driving conditions, whereas cloud gaming becomes a continuous stream of adaptation with user inputs [38]. However, UAV-based surveillance [11] [12] systems also experience some of these sudden surges in data transmission due to environmental factors.

Dynamic tasks raise unpredictability and several hurdles into consideration-real-time processing constraints, workload variations, and potential network congestion. Researchers have attempted to tackle these issues from several different perspectives on adaptive resource allocation: summoning computational power, bandwidth, and memory on a dynamically changing scale by the primary requirement [39]. In contrast, task segmentation is a miracle; one large task of computation is segmented into smaller tasks distributed over multiple processing nodes for effective processing. In addition, data remnant removal tools can provide additional resource optimization by filtering unnecessary transmissions and hence providing extra performance to the system [40]. The table 2, showing the comparative analysis of static and dynamic task offloading:

D. TASK FORMULATION

Task offloading modeling refers to the definition of mathematical models in which the computation task is to be executed. Generally, this is some form of optimization problem involving different constraints and objectives, such as energy efficiency, minimizing latency, and resource allocation [41], [25], [35], [26], [1]. Some of the important steps involved in task offloading formulation are given in equation 1:

$$T_i = (P_i, D_i, C_i, L_i,) \quad (1)$$

Where the components for task offloading are: priority of the task (P_i), input data size required for processing task (D_i), computational complexity, measured in CPU cycles (C_i), and latency or time required for execution (L_i). For the input parameters-based tasks offloading, you can refer to Table 3 for the comprehensive overview. When deciding about the offloading, static tasks (those with well-defined computation requirements) and dynamic tasks (that change with workload and resource needs) become decisive in workflow [35].

1) Why do we want to offload a task?

Task offloading has emerged as a critically important reason for computing systems to improve their performance, efficiency, and resource utilization within the entire architecture. Here are some reasons why task offloading is desirable:

- Reducing the computational load on user devices: Mobile and IoT devices usually have very limited computing power capabilities. Whenever complex work is run locally, the processor of these devices is strained; subsequently, it does not perform effectively, and the idle turns off the performance or fails [42].
- Minimizing Energy consumption: Processing-intensive tasks usually drain batteries much more quickly, which is a very crucial issue for mobile devices and embedded devices. Reduced local processing, therefore, creates energy savings by offloading these tasks to an external server, which will increase the lifetime of battery use [43].
- Low latency in real-time application: Some applications, such as autonomous driving, authenticity, and gaming in the cloud, require real-time processing with the least amount of latency between user interaction and application action. Edge Computing and Multi-Access Edge Computing (MEC) mitigate latency effects by collecting various processing tasks close to the user rather than relying on distant cloud servers [42].
- Advanced Scalability: Scalability is increased through offloading because workloads are spread over multiple remotely located processing nodes (RPNs). This guarantees that increasing computational demands can now be met without the need to upgrade end-user devices [44].
- Allow Complex AI and Machine Learning Tasks: AI and all its applications in image recognition, natural language processing, and data analytics require high

TABLE 2. Comparative analysis of static and dynamic task offloading

Feature	Static Tasks	Dynamic Tasks
Predictability	High	Low
Resource Allocation Complexity	Low	High
Latency Sensitivity	Medium	High
Energy Consumption Impact	Predictable	Varies significantly
Optimization Complexity	Low	High
Use Cases	Batch processing, image analysis, video transcoding	Autonomous driving, cloud gaming, real-time analytics

TABLE 3. Tasks offloading based on input parameters

Input Parameters				
Priority of the Task	Input Data Size	Computational (CPU cycle)	Complexity	Latency Required for Parameter
[64], [71], [33], [35], [54], [4], [56], [55], [108], [107], [11], [16]	[42], [70], [59], [64], [76], [71], [33], [35], [54], [4], [34], [56], [55], [109], [108], [107], [110], [11], [6], [16], [12], [8]	[42], [70], [59], [29], [64], [76], [71], [33], [35], [54], [4], [34], [56], [55], [109], [108], [107], [110], [11], [111], [6], [16], [12], [8]		[64], [76], [71], [33], [35], [54], [4], [34], [56], [55], [108], [107], [110], [11], [16]

computational resources. The offloaded workload ideally will execute on the cloud or edge AI models, making it easy to bring intelligent applications to resource-constrained devices [45].

- **Optimizing Network and Storage Resources:** Deep-diving is not only about cutting down on the expenditure of how much data is transmitted. It also frees the network congestion by processing and gathering at the edge or fog computing environment, at times when it would make sense to transmit these findings over a network.
- **Improving Reliability and Fault Tolerance:** In the name of fault tolerance and redundancy mechanisms, highly remote servers and cloud environments tend to have more of such in comparison to transfer mobility. Offloading allows task performance impossible by the user device due to failures such as network disconnection and hardware malfunction [46].
- **Facilitating Multi-Device Collaboration:** It is possible to synchronize IoT devices, robots, drones, and all the interconnected systems for mingling easily through distributed computing environments. Distributed computing environments where different activities can be done from different Node Points [47].

E. CHALLENGES IN OFFLOADING A TASK

For a comprehensive overview of challenges in computation offloading a task, you can refer to figure 7 and table 4.

- **Network Latency and Bandwidth Limitations:** One of the main challenges involved in computing offloading is network latency and bandwidth limitations. The two critical factors in task offloading to remote processing nodes(RPNs), such as cloud or edge servers, are the speed and reliability of data transmission. Offloading

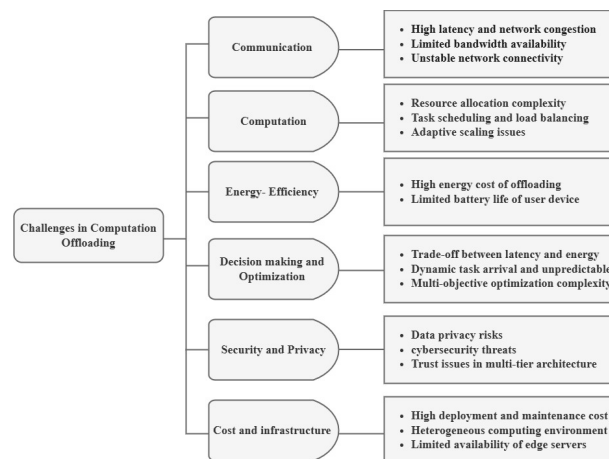


FIGURE 7. A comprehensive classification of challenges in computation offloading.

does not work in real-time applications such as cloud gaming and autonomous driving due to high latency and very low task bandwidth for tasks, which may lead to consequent delays in executing tasks. Network congestion also worsens the situation, especially when many devices offload tasks to a particular server at once [48].

- **Resource Management and Distribution:** Resource management and resource distribution are crucial issues. Intelligent resource allocation algorithms are essential for the efficient distribution of computational tasks among all RPNs. The exact approach to be offloaded is hard to define due to varying workloads and different processing capabilities. Moreover, there have been

TABLE 4. Analysis based on challenges while assigning a task

Challenge			
Network latency and bandwidth limits	Privacy and security risk	Energy consumption	Task dependency and execution constraints
[62], [63], [58], [59], [29], [64], [74], [65], [76], [79], [79], [71] [80], [73], [112], [33], [54], [4], [113] [107], [110], [11], [114], [19], [16], [20] [12], [10], [44], [46]	[63], [58]	[62], [63], [58], [59], [29], [64], [74], [65], [106], [76], [78], [71] [75], [80], [73], [112], [33], [54], [4] [34], [56], [113], [55], [109], [107], [110] [11], [111], [114], [16], [20], [12], [10] [7], [46], [47]	[58], [59], [64], [76], [71], [75], [80], [73], [33] [4], [56], [55], [109], [109], [107], [114] [19], [20], [12]

attempts in the literature to use reinforcement learning techniques to distribute resources dynamically in such a way that the workloads are balanced and efficiency is optimized [49].

- **Privacy and Security Risks:** Privacy and Security Risks are another obstruction to compute offloading. Sensitive data can pose threats such as man-in-the-middle, rather than permitted access, and data breaches while being transferred to an external system. Hence, one must have secured communication and proper, stringent authentication protocols in effect, creatively so as to mitigate these risks [50].
- **Energy Consumption:** In addition, energy consumption in mobile devices and IoT devices remains a drawback. Although offloading is aimed at alleviating the computational burden from the user's equipment (UE), poorly executed decisions either increase the cost or, worse, lead to an increase in it. Research indicates that optimized task partitioning and usage of energy-aware offloading techniques lead to effective power savings [51].
- **Task Dependencies and Execution Constraints:** Lastly, added complications arise from task dependencies and execution constraints. For instance, some tasks may often be interdependent, and thus partial offloading may not be possible. This becomes more complicated in multi-tier architectures as they hold tasks between the cloud, edge, and fog computing layers. Creating adaptive task scheduling algorithms considering dependency constraints would improve the overall performance of a system [52].

V. COMMUNICATION AND COMPUTATION

Two major factors can improve task offloading: communication and computation. Researcher analysis [53], [54], refers to the dependence of the efficiency of offloading on two factors: communication, which includes data transmission, and computation, or energy consumption, depending on the process power use. Communication-based analysis involves analyzing different parameters that influence offloading in terms of networks, such as bandwidth, transmission time, and network latency, as illustrated in Table 5. Computation-based analysis involves checking how efficiently applications

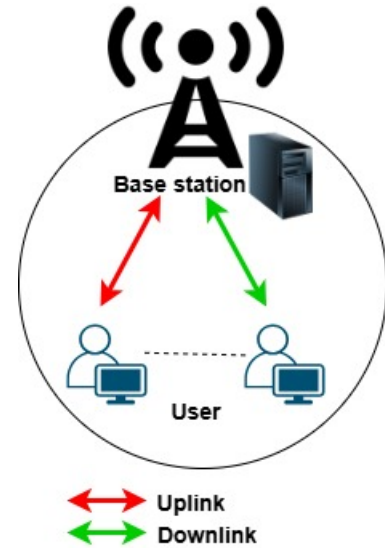


FIGURE 8. Communication flow between users and base station in computation offloading

TABLE 5. Communication and communication-based analysis

Communication and computation	
Communication parameter based	Computation parameter based
Transmission time, network latency, Bandwidth, and Energy consumption for transmission	Execution time, Processing power, Energy consumption for computation, Task completion time, and Resource utilization
[42], [63], [58], [59], [29], [79], [71], [80], [73], [54], [4], [34], [113], [55], [108], [107], [110], [11], [111], [6], [14], [13], [9], [7], [46], [47]	[42], [62], [63], [70], [59], [29], [64], [74], [65], [106], [76], [53], [78], [79], [71], [75], [80], [112], [33], [54], [4], [72], [34], [56], [55], [109], [107], [110], [115], [11], [111], [114], [6], [19] [16], [14], [15], [12], [10], [7], [8], [44], [46], [28]

are processed at remote nodes like cloud or edge servers, as depicted in Figure 9. The performance of offloading in communication relies on the speed at which data travels from the user device and equipment (UE) to the respective remote processing node (RPN). The communication flow between users and the base station during compute offloading, as

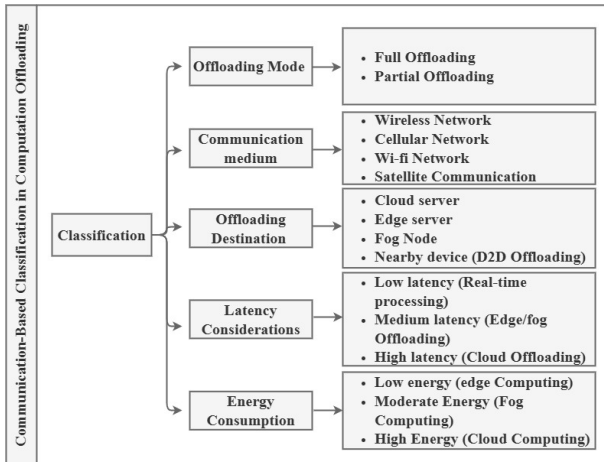


FIGURE 9. Categorization of computation offloading based on communication aspects.

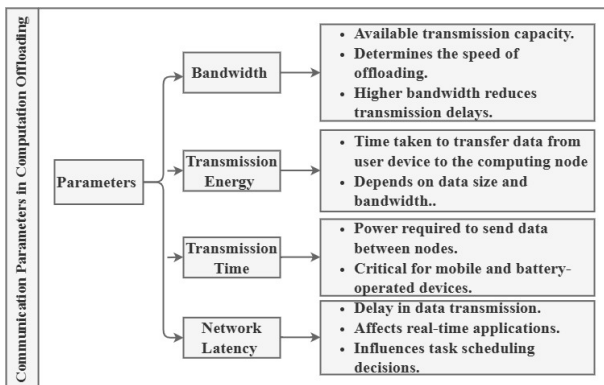


FIGURE 10. Key communication parameters in computation offloading.

portrayed in Figure 8, reveals the accompanying uplink and downlink communication. Therefore, the higher the amount of data considered for offloading that gets processed and transmitted, the less the time to transmit the information [54]. In computation analysis, the focus is on the degree of processing involved at the offloading node and how the tasks are processed at the offloading node after their arrival at there node [55]. This computation offloading classification that explains, in detail, offloading techniques, latency-related issues, and energy consumption issues is illustrated in Figure 10. On the other hand, Figure 11, introduces the communication characteristics affecting the nature of offloading at such parameters, such as transmission time, energy, and bandwidth. The computation parameters include task complexity, resource availability, latency, and cost analysis for the assurance of an optimal offloading choice process.

VI. RESOURCE ALLOCATION AND DECISION MAKING

Resource allocation in computational offloading is one of the most important ways to optimize execution while simultaneously ensuring better QoS. While a task is offloaded to remote processing nodes (RPNs), such as cloud, edge, or fog computing infrastructure, it must ensure the appropriate

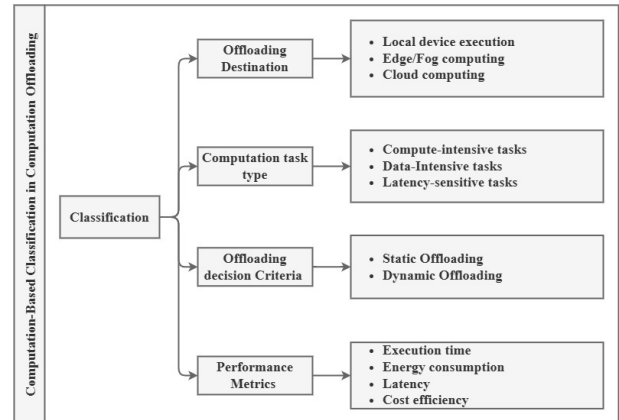


FIGURE 11. Key factors in computation-based offloading decisions.

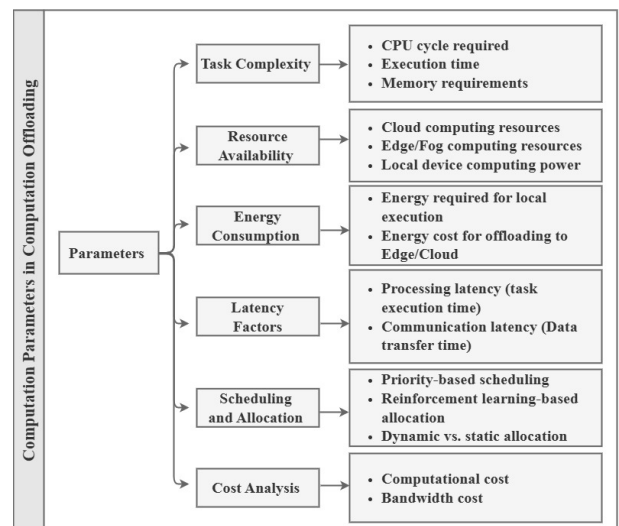


FIGURE 12. Key computation parameters in offloading decisions.

amount of computational resources from a pool of CPU cycles, memory, and bandwidth [11], [16], and [12]. Inefficient resources would, on the one hand, cause delays in processing due to inefficient utilization and excess power consumption, and on the other hand, result in poor performance of the complete system; hence, there must be an allocation based on the instant demand scenario. To improve the efficiency of the offloading procedure [56], some scheduling techniques can be implemented. Priority scheduling [57] gives someone resource-cropping capabilities to more urgent tasks first. This ensures that applications with stringent latencies benefit from this option for priorities. Resource allocation and data exchange in edge computing are demonstrated in Figure 13, which illustrates a multi-user edge computing system where user devices offload computation tasks to an edge server. Different types of data flow are indicated by arrows of different colors: the green, red, and purple arrows represent environmental data collection, offloading links, and instruction feedback. Resource forecasting for system demands and decision making are enhanced by environmental

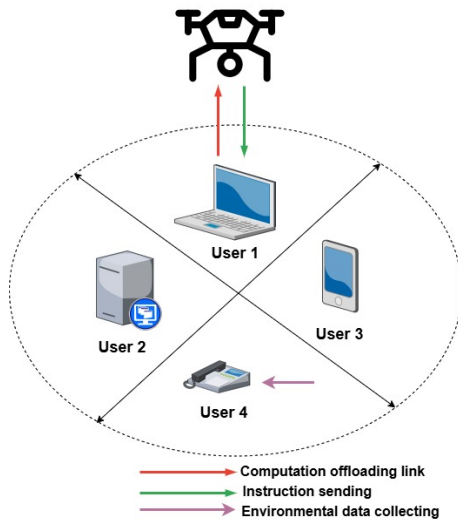


FIGURE 13. Resource allocation and data exchange in edge computing.

data collected through reinforcement learning algorithms, federated learning, deep supervised learning, and unsupervised learning models. All of these AI models constitute a single framework that dynamically adapts and integrates all the processes for adaptive resource allocation and computation offloading in edge computing environments. Complementing this, Figure 14 complements this by detailing the resource allocation process spanning task generation, classification, allocation layers, techniques, and optimization metrics, where these AI models automate for task generation, task classification, forecast execution needs, and select adaptive allocation strategies. These architectures form an integrated, intelligent framework for efficient resource allocation and decision-making in edge, fog, and cloud computing systems. Allocation of resources based on reinforcement learning [58], [59], and [60] employs AI models to predict and optimally allocate resources in a Multi-Access Edge Computing (MEC) environment. An adaptive method for resource management helps improve the execution of tasks by dynamically varying resource allocations based [52] and [38] on fluctuations in workloads. Computational offloading using intelligent resource allocation schemes will lead to higher efficiency and balanced workload distribution, hence becoming a major necessity for next-generation computing framework designs [40] and [32]. Decision-making is the last step in computation offloading, and this is when the system decides the best way of executing the task according to the previously defined parameters. If the offload task completes successfully, the output data is returned so that the original device can be used. Whenever network conditions weaken or system constraints change, the offloading decisions should be dynamically re-evaluated. Decision-making may also involve a machine learning-based model to optimize future offloading strategies for improving efficiency and enhancing overall performance for mobile and embedded systems [61].

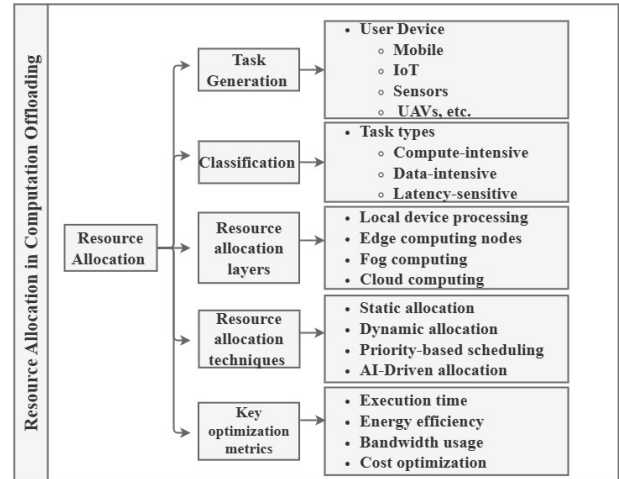


FIGURE 14. Resource allocation framework in computation offloading.

A. COMPUTATION OFFLOADING USING:

- 1) Basic Supervised Learning
- 2) Unsupervised learning algorithms
- 3) Mathematical modeling
- 4) Deep learning models
- 5) Reinforcement learning
- 6) Federated learning
- 7) Blockchain-based offloading
- 8) Using domain knowledge on the historical data

1) Basic supervised learning approaches:

In our literature review, we find that a large amount of prior offloading research was focused on supervised learning techniques. Initially, we mention the most pertinent research, followed by Table 7. Minimizing delays is one of the typical targets for representation optimization that has been investigated in most of the previous work. Shuai and Wang et al. [62] propose a framework that utilizes Knowledge-Defined Networking (KDN) and machine learning to optimize computation offloading in resource-constrained environments, such as disaster response scenarios. By creating a "Hyperprofile" that represents edge nodes in a multidimensional feature space based on predicted matrices, the framework employs k-nearest Neighbor (kNN) queries to identify optimal nodes for task offloading. Huaming and Knottenbelt et al. [63] proposed that the min-cost offloading partitioning (MCOP) technique is the main focus of an effective application partitioning algorithm for mobile contexts. To distribute tasks between mobile devices and cloud/edge servers as efficiently as possible, it uses a profile to gather information about devices and network parameters. By reducing runtime and cost, the MCOP algorithm improves performance in response to changes in network conditions. Supervised Learning is a type of machine learning in which models are trained on labeled datasets. With respect to computation offloading, supervised learning can be predicted based on several system parameters, regardless of whether the task is locally offloaded. Use

case of supervised learning in computation offloading:

- Offloading decision making: Compute whether to transfer a task to a cloud or edge server.
- Resource allocation: Explore the best possible allocation of memory, CPU, and bandwidth.
- Latency predictions: Estimate execution time and network latency before deciding where to carry out the tasks.

Yaping and Liang et al. [64] proposed a resource allocation order to enhance system performance and dynamically manage computing resources. This study suggests an intelligent task of obtaining a mobile edge computing (MEC) algorithm from a vehicular network. This technology uses reinforcement learning to solve resource allocation and K-nearest neighbors(KNN) methods to select the best offloading platform (local, MEC server, or cloud server). Comparing the suggested methodology to baseline techniques like "Full MEC" and "Full Local" the system cost can be reduced by up to 80%

Computation offloading based on supervised learning deals with problems that determine whether a task is being processed on the local device as a classification or a regression problem. Therefore, problems can be formulated mathematically as in functions given in eq2:

$$f : X \rightarrow Y \quad (2)$$

where, X is the collection of input features, representing system parameters, and Y is the output that indicates the offloading decision. The input features usually include task size (s), computational requirement (C), network conditions such as bandwidth and round-trip time (BW, RTT), device energy level (E), and workload of the server (Q). The output labels could typically be structured as a binary classification problem, where $Y = 1$ indicates "offloading is beneficial"; in contrast, $Y = 0$ implies local execution. As shown in Table 6, Several supervised learning models are applied to train offloading decisions, and Figure 15, illustrates a supervised learning-based framework for optimized task offloading in a computation offloading system. Input parameters such as application task type, CPU cycle, task size, bandwidth, and data size are processed by machine learning models, including SVM, KNN, linear regression, and logistic regression. These models learn from historical data to predict optimal offloading decisions, aiming to minimize energy consumption and latency. In the system architecture, supervised models operate at the edge or cloud server, continuously updating decision policies based on new task data, enabling precise and adaptive resource allocation in dynamic network conditions.

Ayesha Abdul and Khan et al. [65] proposed a basic base classifier (support vector machine) used to adapt code offloading methods to make accurate and dynamic offloading decisions in mobile cloud computing systems. The system profiles devices, applications, and network properties to determine whether tasks are run locally on the device or offloaded to a remote server. A 92% classification accuracy

TABLE 6. Table for the approaches for supervised learning approaches

Algorithm			Use Case in Computation Offloading
Decision Trees (DT)			Learning simple offloading rules based on conditions
Random Forest (RF)			Handling complex multi-feature offloading scenarios
Support (SVM)	Vector	Machines	Classifying offloading vs. local execution
Artificial (ANNs)	Neural	Networks	Learning complex relationships between system parameters
Deep Learning (DNN, CNN, LSTM)			Handling dynamic, real-time offloading decisions

is achieved by the proposed approach, which significantly lowers response time and energy usage. While the classifier is regularly updated to adjust to changing situations, the modular and adaptive design guarantees real-time decision-making.

2) Unsupervised learning approaches:

This section describes previous research on offloading. This focuses on unsupervised learning techniques. These studies are summarized in Table 9. A review of the literature shows that Unsupervised learning techniques were exposed to learning techniques compared to supervised learning techniques. Most of the studies used in this section employed clustering techniques. In this case, one of the significant factors delaying the system is interference between users. Diao et al. [66] implement a thorough optimization scheme to resolve problems that reduce system delay through user clustering, resource allocation, and simultaneous treatment of power supply, which is well known to be non-convex. They also use particle swarm optimization (PSO) for iterative control of power. As per simulation results, the proposed NOMA-based MEC system performs better than Orthogonal Frequency Division Multiple Access in highly populated user situations with low numbers of users in terms of latencies and spectrum efficiency improvements. Another study by Ivanchenko et al. [67] uses the hidden semi-Markov model (HSMM) to search for alternative routes based on the available power. Several machine learning methods, as shown in Figure 16, present an unsupervised learning- based framework for optimized task offloading, where such as application task types, CPU cycles, task sizes, bandwidth, and data sizes are processed without labeled outcomes. Models such as PCA and clustering group tasks with similar characteristics, NLP interprets task-related metadata, and Markov models predict state transition in resource availability or network conditions. Within the model's system architecture, these models operate on the edge or

TABLE 7. Comparison of various approaches of supervised learning in computation offloading.

Ref	Problem	Model	Parameters Considered for Decision Making	Result	Scope of improvement
[63]	Challenges in application partitioning under variable network conditions addressed.	KNN Model, Partitioning Algorithm	Response time, energy consumption, network status, and device characteristics.	Dynamic partitioning algorithm improves response time and energy efficiency.	Improve efficiency, adaptability, and energy strategies for application partitioning.
[62]	Optimizing computation offloading in mobile edge networks efficiently.	KNN Model, Linear Regression	E and L	Optimize decisions based on energy or latency requirements of the user.	Enhancing adaptability and efficiency in computation offloading frameworks.
[64]	Minimize vehicle delay costs in a constrained vehicular network.	KNN Model	Task size, CPU cycles, maximum tolerable delay, transmission power, channel gain, and computing resource allocation.	The Proposed algorithm effectively reduces delay in vehicular computing tasks.	Future work includes optimizing energy consumption and multi-server resource allocation.
[65]	Dynamic computation offloading decision-making for mobile systems' efficiency.	Support Vector Machine (SVM)	Local execution time, remote time, CPU usage, bandwidth, data size.	The Proposed system achieves high accuracy in dynamic offloading decisions.	Improvement in energy efficiency and performance for mobile applications.
[112]	Dynamic computation offloading challenges in edge computing, optimizing cost and performance for IoT.	Dynamic Computation Offloading Algorithm (DCOA)	System Parameters, Task Parameters, Cost Parameters, Optimization Parameters.	DCOA optimizes offloading, balancing cost and performance effectively.	Dynamic task execution and resource allocation optimization in edge computing.

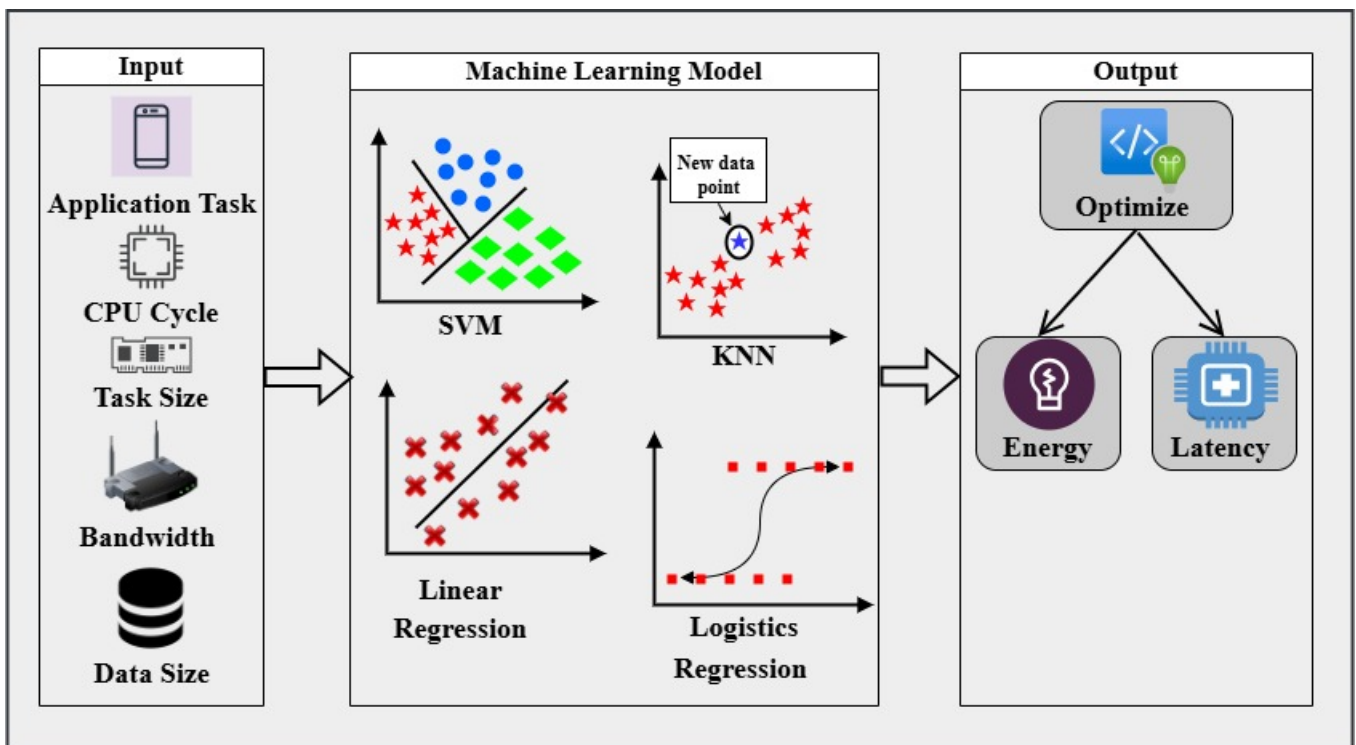


FIGURE 15. Supervised learning approach for optimized task offloading in computation offloading.

cloud server to uncover hidden patterns, enabling adaptive task grouping and resource allocation that minimizes energy consumption and latency in dynamic environments. Some of the critical roles of unsupervised learning in enhancing system efficiency and adaptability include applying techniques such as:

- Task Classification and Clustering: Grouping similar computation tasks for optimum resource allocation.
- Anomaly Detection: Detecting anomalies in workload spikes or failures in offloading decisions.
- Network Traffic Optimization: Assessing usage patterns to optimize bandwidth and reduce network congestion.
- Energy & Load Balancing: Grouping devices or tasks based on resource consumption for higher efficiency.

Mathematical framework:

- 1) The basic clustering Mapping: Unsupervised learning entails discovering hidden patterns from an unlabeled dataset of instances. It can be expressed mathematically as follows: equation 3

$$f : X \rightarrow Cluster(C) \quad (3)$$

Where X associates the input dataset with features such as task size, CPU workloads, network delay, bandwidth, and energy consumption, and C refers to clusters or groups formed based on the similarity between data points. The data set is composed of multiple input features important for describing the task requirement and system condition, forming the dataset for the determination of computation offloading decisions. Each task is represented as a feature vector in the form of an equation 4:

$$X_i = [S, C, BW, RTT, E, L] \quad (4)$$

where S means task size or amount of data to be processed, C refers to computational requirement in terms of CPU cycles, BW (Bandwidth), and RTT (Round-Trip Time) define the network conditions, such as available bandwidth and round-trip time. As for E, it is the energy consumed by mobile devices, whereas L represents the entire perceived delay, including execution and communication delays. Several unsupervised learning methods are applied in computation offloading.

- 2) Distance metric (Used to form clusters): Clustering algorithms, such as K-Means and hierarchical clustering, require a method to quantify the similarity between two data points. The most frequent is the Euclidean distance, which is shown in the equation 5:

$$\text{dist}(Y_i, Y_j) = \sqrt{\sum_{k=1}^n (x_{ik} - x_{jk})^2} \quad (5)$$

This equation computes the distance between two tasks Y_i and Y_j based on their features.

TABLE 8. Unsupervised learning algorithm for computation offloading

Algorithm	Uses in computation offloading
K-Means Clustering	Grouping tasks based on resource requirements.
Hierarchical Clustering	Organizing tasks and network nodes for adaptive load balancing.
Gaussian Mixture Models (GMMs)	Probabilistic modeling of offloading decisions.
Principle Component Analysis (PCA)	Reducing dimensionality for efficient offloading decision making.
Autoencoders	Feature extraction and anomaly detection in offloading strategies.

- 3) K-Mean clustering: K-Means is commonly employed for clustering tasks. Its purpose is to reduce the variance among the clusters and the presentation in the equations 6:

$$K = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|^2 \quad (6)$$

Where, x is a data point (e.g., one task with features like size, delay, energy), k is the number of clusters, μ_i is the centroid (center) of cluster C_i , and $\|x - \mu_i\|$ is the squared distance between point x and the cluster center μ_i . The function minimizes the sum of squared distances between each point and its cluster center.

- 4) PCA (Principal component analysis): Principal component analysis, or PCA, is a data transformation technique used for dimensionality reduction, a process that reduces data in higher dimensions and transforms them to fewer dimensions that still capture the most variance. PCA transforms the original feature matrix Y into a lower-dimensional representation X by projecting it onto a set of principal components Z , and the transform is shown in the equation 7:

$$X = YZ \quad (7)$$

Unsupervised learning constructs a powerful framework to shape the optimization of computation offloading by grouping tasks, identifying anomalies, and enhancing load balancing. Samir et al. [68] proposed a framework for monitoring, detection, and localization of performance anomalies in container-based clusters by using the hierarchical hidden markov model (HHMM). Therefore, it's an attempt to identify the basic cause of the terminology to ensure maximum availability and performance of the system can be used. K-means clustering, principal component analysis (PCA), and hierarchical clustering technology. Shang et al. [69] proposed an advanced computational scheme based on the AHP-analytic hierarchy process. AHP is the technology that takes

into account both the weights and simultaneously optimizes the objective function. AHP is the structure of an analytical solution to a complex layer of problems. As shown in Table 8, these techniques develop dynamic adaptive decision structures that function to ensure the effective use of available computing resources in edge and cloud environments.

3) Mathematical Modeling

This section describes previous research on computation offloading, with a primary focus on mathematical modeling techniques. Table 11 provides a summary of these studies. Mathematical modeling of the computation offloading optimization problem with machine learning. This determines whether tasks are performed on local devices or whether they are offloaded to edge or cloud servers. The objective typically is to maximize performance indicators such as energy consumption, latency, or network utilization while enabling efficient resource allocation. Zaiwar and Jiao et al. [70], the Energy-Efficient Deep Learning-Based Offloading Strategy for Mobile Edge Computing (MEC), develop a comprehensive cost function that allows intelligent offload decision-taking into account energy consumption, network conditions, and computing load. Comparing EEDOS to current approaches, there is a 3% to 10% decrease in energy consumption and a 12% increase in DNN accuracy. Although this approach works well in cases involving a single user, the multi-user steps and nonlinear execution models require further research and development. Jing and Yuan et al. [71] suggested that genetic simulated annealing-based particle swarm optimization (GPS) is an energy-optimized particle computation offloading strategy for mobile edge computing (MEC) systems. By optimizing offloading ratios, CPU frequencies, transmission power, and bandwidth allocation while adhering to latency limitations, the method reduces the total energy consumption of intelligent mobile devices and edge servers. GPS is a scalable solution for dynamic MEC contexts such as IoT and 5G networks and was validated using real-world data to surpass existing algorithms in terms of energy efficiency and convergence rate. Below are the 6 steps of mathematical representation used in this computation offloading context:

- 1) **Task Representation as a feature vector:** Each computation task X_i is represented as a vector of features in the form of an equation 8:

$$X_i = [S, C, BW, RTT, E, L] \quad (8)$$

where S means task size (bits), C refers to computational requirement in terms of CPU cycles, BW (Bandwidth), and RTT (Round-Trip Time) define the network condition, such as available bandwidth (Mbps) and round-trip time (ms). As for E, it is the energy consumed (Joules) by the mobile devices, whereas L stands for total latency (ms). The set of these features serves as the major input for machine learning models, which are used for the classification or clustering of tasks.

- 2) **Offloading Decision Variable:** For task offloading or local processing, a binary decision variable O_i is defined as in the form of an equation 9:

$$O_i = \begin{cases} 1, & \text{if task } i \text{ is offloaded} \\ 0, & \text{if task } i \text{ is executed locally} \end{cases} \quad (9)$$

The information needed to ascertain the value of O_i lies in the use of a machine learning classifier (Decision Tree, Neural Network, SVM, etc.) with certain task parameters.

- 3) **Latency Modeling:** The total latency L_i of a task includes computation time, transmission time, and queuing delay, and can be represented by the equation 10:

$$L_i = L_{\text{comp},i} + L_{\text{trans},i} + L_{\text{queue},i} \quad (10)$$

where:

- **Computation Time:** if executed locally, $L_{\text{comp},i}$ can be calculated using the equation 11, as follows:

$$L_{\text{comp},i}^{\text{local}} = \frac{C_i}{F_{\text{local}}} \quad (11)$$

where, F_{local} the CPU frequency frequency of the local device. If offloaded, the computation time can be calculated using the equation 12, as follows:

$$L_{\text{comp},i}^{\text{cloud}} = \frac{C_i}{F_{\text{cloud}}} \quad (12)$$

where F_{cloud} is the CPU frequency of the cloud/edge server.

- **Transmission Time:** The time required for data transmission over a network can be determined using the equation 13, as shown below:

$$L_{\text{trans},i} = \frac{s_i}{BW} \quad (13)$$

- **Queuing Delay:** There are instances where the task may be delayed in the server queue, which can be calculated using the equation 14, as shown below:

$$L_{\text{queue},i} \approx \frac{N_{\text{queued}}}{\mu} \quad (14)$$

where N_{queued} is the number of tasks in the queue, and μ is the rate of processing at the server.

- 4) **Energy consumption modeling:** The total energy consumption of a mobile device, including both localization versions and transmission energy when offloaded, can be determined using the equation 15, as shown below:

$$E_i = E_{\text{comp},i} + E_{\text{trans},i} \quad (15)$$

where:

- **Local execution energy** calculated as equation 16:

$$E_{\text{comp},i} = \kappa C_i F_{\text{local}}^2 \quad (16)$$

where κ is the energy coefficient for the processor is represented by κ .

TABLE 9. A synopsis of unsupervised learning research in computation offloading.

Ref	Problem	Model	Parameters Considered for Decision Making	Result	Scope of improvement
[66]	Improved number of user accesses and spectral efficiency	Hybrid (Clustering, PSO)	Energy and latency	Minimize energy consumption, power supplies, user terminology, communications, and computing resources	Single micro cell assumption
[69]	Improving the user experience	AHP (Analytic Hierarchy Process)	Delay- QoE- Speed- Bandwidth- Time- Energy	Reduce service delay-reduce energy consumption multi-user, multi-server	High complexity
[68]	Monitor, detect, and localize performance anomalies in container-based clusters	HMM (Hidden Markov Model)	Delay, energy, response time	Recognize and identify abnormal behavior of containers and node workloads	Not considering more anomaly scenarios
[116]	Multi-objective optimization	HMM (Hidden Markov Model)	Energy and latency	Identifying the optimal system configurations for various requirements and minimizing total energy consumption	Need to take into account potential solutions and address the issue
[67]	Assessing and adjusting the IaaS cloud's overall performance level during failure	HSMM (Hidden Semi-Markov Model)	Availability, Reliability, and QoS	Get a general equation in embedded DTMCs	Need to focus on the specification of numerical parameter values for modeling and creating web-oriented platforms

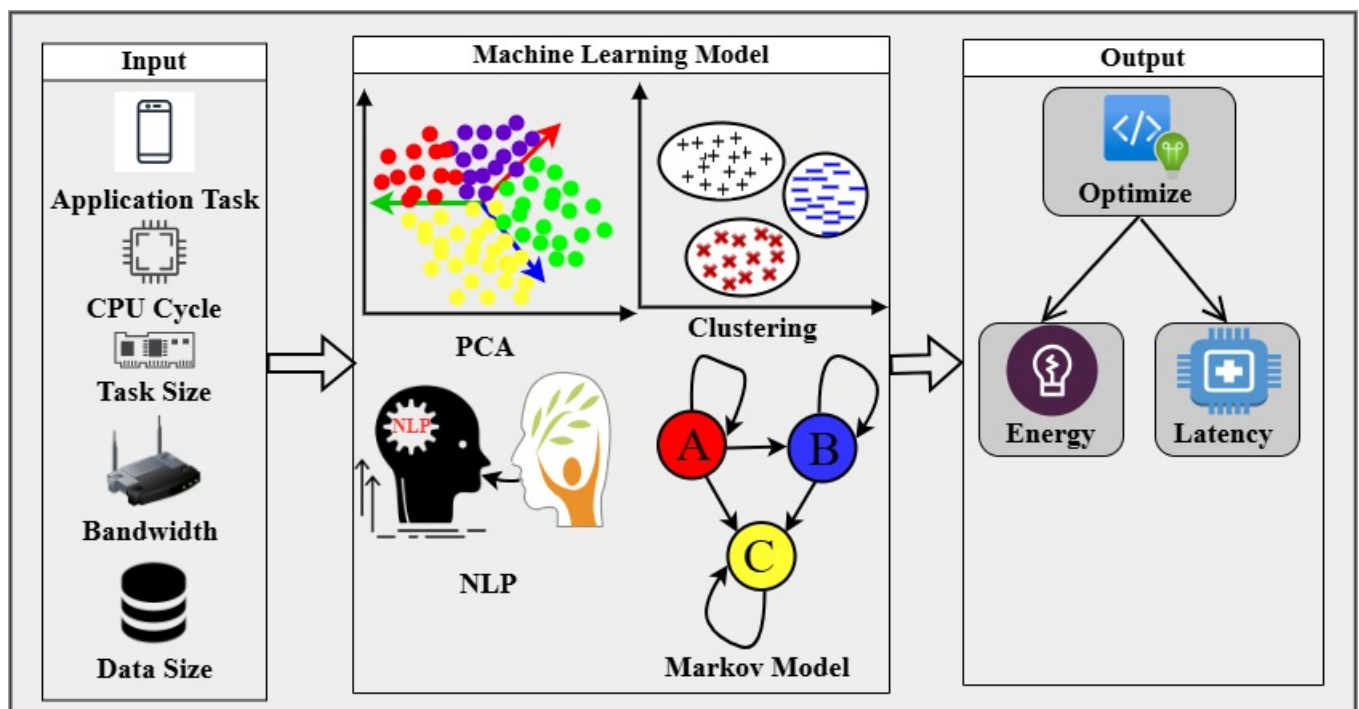


FIGURE 16. Unsupervised learning approach for optimized task offloading in computation offloading.

TABLE 10. Machine learning approach for computation offloading

Machine Approach	Learning	Examples
Supervised Learning		Decision Trees, SVMs, Neural Networks
Unsupervised Learning		K-Means Clustering, Hierarchical Clustering
Reinforcement Learning		Deep Q-Network (DQN), Policy Gradient Methods

- Transmission energy calculated as a equation 17:

$$E_{\text{trans},i} = P_{\text{tx}} \cdot L_{\text{trans},i} \quad (17)$$

where P_{tx} signifies the power consumption of the communication module.

- 5) **Optimization Objective:** The computation offloading process is typically defined by an objective function, which is a weighted sum of energy and latency, as determined using the equation 18, as shown below:

$$\min_{O_i} \sum_{i=1}^N (\alpha E_i + \beta L_i) \quad (18)$$

whereby the weighting coefficient are given by α and β . Heuristic algorithms (e.g., Genetic Algorithm, Particle Swarm Optimization) or machine learning models, such as reinforcement learning and deep learning, are then used for optimization.

- 6) **Machine learning approach:** The Machine learning algorithm can optimize computationally offloaded by adapting the task decision strategies, as shown in Table 10.

Several studies have proposed hybrid solutions for energy-efficient offloading. Researcher Jing and Zhang et al. [33] proposed a hybrid computational paradigm of energy-efficient tasks in mobile edge cloud systems. This focus on both static and dynamic applications. They introduce the particle swarm optimization based on the genetic learning algorithm (PGL), resource usage and energy consumption in the final period. Experimental results show that PGL is superior to conventional technology in terms of energy efficiency and convergence rate. In the other studies Joseph Henry and Yue et. al. [54], reduce energy usage while satisfying latency limitations, the study efficiently optimizes CPU frequency modifications, transmit power allocation, and offloading ratios. Ke and Mao et.al. [4] proposed a novel energy-efficient computation offloading algorithm (EECO) designed for MEC in a 5G heterogeneous network, which concerns the dual problem of latency limiting and energy consumption. Further, Wen-Jiang and Yang et.al. [72] proposed the EECO algorithm in MEC for 5G networks optimizes offloading together with radio resource allocations thereby

reducing energy consumption at the systems level during high network load times. Besides, various greedy algorithms such as MGA, FGA, and CGA have been implemented to address fairness, efficiency, and resource-aware optimization in multi-user multi-task MEC systems. Songtao and Liu et al. [34] proposed a Directed Acyclic Graph (DAG) model, the eDors algorithm self-optimizes transmission power, CPU clock frequency, and offloading decisions for energy-efficient task scheduling while application deadlines are met. Several studies have been conducted to evaluate the simulation results in different case studies demonstrating the efficiency of these techniques to reduce energy consumption, improve system performance, and make these solutions scalable for the MEC cloud computing environment.

4) Deep learning model

This section discusses previous research on offloading with a major focus on deep learning techniques. These studies are summarized in Table 12. A review of the literature shows that deep learning models from basic artificial neural networks and models such as convolutional neural networks (CNN), recurrent neural networks (RNN), and long-short-term memory (LSTM) are used for decision-making in the context of computation offloading. As shown in Figure 17, a deep learning-based framework for optimized task offloading, where input parameters such as application type, CPU cycles, task sizes, bandwidth, and data size are processed through multi-layer neural networks. Operating at the edge or cloud server, these models learn complex, non-linear relationships between system condition and offloading outcomes, enabling precise prediction of resource requirements. The resulting optimization decision minimizes energy consumption and latency, allowing the architecture to adapt dynamically to fluctuating workloads and network states. It has been observed from the literature that activation functions like ReLU [73] are used for implementing these deep learning model techniques. Shuai and Wang et al. [42], proposed a Deep Supervised Learning algorithm model for offloading the computational task, considering parameters such as energy and latency. To make a decision, it formulates the problem as a multi-label classification problem to minimize system cost, and the neural network (NN) decides which components to offload based on a threshold value of 0.5. Furthermore, a review of Deep neural networks (DNNs) shows that the cost of execution is predicted in computation offloading. Xing and Li et al. [74] proposed a DNNOff, a novel method for offloading intelligent Internet of Things (IoT) applications based on deep neural networks (DNNs) in a mobile edge computing (MEC) setting. DNNOff makes it possible for DNN layers to be dynamic and adaptable across cloud platforms, edge servers, and end devices. The method maximizes reaction time by using a pipe-and-filter design for adaptive offloading and a random forest regression model for execution cost prediction. DNNOff significantly outperformed current techniques in terms of response time, cutting it from 12.4% to 66.6%. According to the literature reviews, the main goal of com-

TABLE 11. Comparison of various approaches in computation offloading for mathematical model.

Ref	Problem	Model	Parameters Considered for Decision Making	Result	Scope of improvement
[70]	Optimal offloading of computational tasks in mobile edge computing.	Mathematical model, DNN Based	Energy consumption, data transfer, network conditions, and computational load are critical parameters.	EEDOS improves accuracy and reduces energy consumption in mobile edge computing.	Improvement focuses on energy efficiency and computational offloading accuracy.
[71]	Minimizing energy consumption in mobile devices through computation offloading optimization.	GSP Algorithm	Energy and Latency	Minimize total energy consumption while satisfying latency constraints, energy limits, and resource availability at the edge server.	Energy-efficient computation offloading in mobile edge computing systems.
[33]	This paper deals with the energy consumption of mobile edge computing for resource-limited devices.	PGL Algorithm	Energy and Latency	Minimize energy consumption while ensuring tasks are completed within their deadline.	Optimizes energy consumption for mobile edge computing through tasks.
[54]	The problem is optimizing task offloading in IoT networks for energy efficiency.	LSCCOA Algorithm	Energy and Latency	Minimize energy consumption while satisfying task deadlines and resource constraints.	Need to improve more energy efficiency and reduce transmission delay in MEC.
[4]	Optimizing energy-efficient task offloading in 5G MEC networks while minimizing energy consumption and ensuring low latency.	EECO Algorithm	Energy and Latency	Ensure energy efficiency and latency compliance.	Optimized energy efficiency, latency, resource allocation, and task offloading in 5G MEC.
[72]	Efficient multi-user multitasking loads at offloading edge cloud optimization costs.	Greedy Algorithms	Latency and Energy	Decision for multi-user, multi-task MEC systems with imbalance edge clouds, minimizing offloading costs.	Optimized offloading, cost reduction, fairness, resource efficiency, scalability, and dynamic edge adaptation.
[34]	Energy-efficient computation and task scheduling in mobile cloud computing.	Dynamic Offloading	Energy and Latency	Balance energy consumption and task completion time dynamically.	Optimized energy efficiency, task scheduling, offloading decisions, resource allocation, and execution time reduction.
[117]	Assign workflow sub-tasks to edge nodes while minimizing resource usage	Cuckoo search algorithm, Particle swarm optimization	Energy, Latency, and resource utilization.	QIPSO algorithm outperforms others in workflow scheduling efficiency.	future studies may address fault tolerance and preemption issues.

putational research is to know whether tasks are performed locally on mobile devices, edge servers, or clouds for optimal performance of the system metrics. It can predict that the task is executed in a certain time, rounding off with network congestion and how much energy is consumed for the purpose of intelligent task allocation. There are some deep learning-based computation offloading models:

- **Supervised Deep Learning Models:** In these approaches, historical actual data have been used to train deep neural networks so they can predict offloading decisions based on task size, computation requirement, available band-

width, and latency constraints.

- **Unsupervised Deep Learning Approaches:** Examples of clustering-based deep learning methods, including autoencoders and self-organizing maps, define the classification of a task into dissimilar offloading types without requiring any labeled training data.
- **Reinforcement Learning Techniques:** Offloading decisions may, therefore, be dynamically improved using DQNs and Actor-Critic models, which learn the most effective offloading policies through reward signals derived from the real-time operational performances.

Deep learning-based computational exposure is formulated as an optimization algorithm whose objective function is to minimize energy consumption and latency. The binary offloading decision O_i is expressed in equation 19:

$$O_i = f_{\theta}(X_i) \quad (19)$$

where, f_{θ} is a neural network defined with trainable parameters θ . The output of the model O_i , which is a binary decision variable, indicates whether or not the task i is to be offloaded. The objective function for the loss function optimization deep learning model is trained to minimize an equation that balances consumption and latency, and it can be calculated using the equation 20, as shown below:

$$\min \sum_{i=1}^N (\alpha E_i + \beta L_i) \quad (20)$$

where α and β are weighting coefficients. E_t is energy consumption and L_t is task latency. The optimization strategies for task offloading of deep neural networks (DNNs) from end-users to the MEC system, known as a DNN-MEC, have been discussed quite extensively. Mingjin and Cui et al [53] applied mixed-integer linear programming (MILP) technique, which suggested a framework that reduces the computational load on mobile devices and minimizes processing latency for DNN tasks. The MILP-based optimization mechanism efficiently determines the best offloading and task partitioning techniques using this model. Additionally, Yongsheng and Lv et al [75] proposed the energy and performance efficient deep learning-based offloading algorithm (EPED), which dynamically selects the best offloading choices. EPED reduces execution time, energy consumption, and server resource utilization by decomposing applications into their parts and creating a thorough cost function. In another study, Giha and Kim et al. [73] proposed a DNN-based object detection offloading system for mobile edge devices to improve average frames per second (FPS) by effectively allocating workloads between devices and edge servers. It is proposed that greedy binary offloading decisions are maximized when considering account server queuing delays, device processing power, and network bandwidth. Experimental results show improved performance in real-world WLAN situations. Expanding to application-specific scenarios, Wahida Mansouri and Manal Abdullah Alohali et al. [118] presented a real-time edge-based crowd monitoring framework for smart cities, where rapid detection of crowd density, movement pattern, and potential hazards is critical for public safety. To meet these constraints, offloading to nearby MEC or fog nodes is preferred over cloud, reducing transmission delay and enabling rapid decision-making. Such requirements shape architectural choice, resource allocation, and adaptive policies. Similarly, Alina L. Machidon and Veljko Pejovi et al. [119] proposed a predictive deep learning framework that enhances resource-efficient system operation by forecasting computational demands and network conditions. Although it does not explicitly address model compression, such as predictive capabilities, they are highly relevant to computation offloading.

By anticipating workload patterns and algorithms, the system can perform preemptive resource allocation and dynamic task migration, ensuring that high-priority or latency-sensitive tasks are processed at the optimal location, whether on the device, MEC, or cloud. Overall, these studies demonstrate how optimization algorithms (MILP), adaptive deep learning models (EPED, DNN-based object detection), application-specific frameworks, and predictive strategies collectively contribute to improving offloading decisions, reducing latency, enhancing energy efficiency, and enabling intelligent workload distribution in an edge computing environment. Future studies can then move to hybrid deep learning and reinforcement learning approaches to contend with increasingly complex multi-device and multi-server scenarios.

5) Reinforcement learning approach

The basic model of reinforcement learning includes many conditions and possible actions. Actions have been taken to change the state of the system, and the overall goal is to put the system in a state that achieves the highest gain. This gain can be interpreted in terms of cost minimization, response time, or maximizing quality of service. As shown in Figure 18, it illustrates a reinforcement learning-based framework for adaptive task offloading, where input parameters such as application type, CPU cycles, task size, bandwidth, and data size define the system state. An agent, operating at the edge or cloud server, interacts with the environment to select optimal offloading action. The interpreter processes system feedback to provide rewards, guiding the agent to improve future decisions. The continuous learning loop dynamically adapts offloading policies to fluctuating network and resource conditions, optimizing both energy consumption and latency within the computation offloading architecture. Some of the studies are summarized in Table 13. There are several approaches to using reinforcement learning to solve computational problems and maximize the quality of service. For the rest of the offloading research, most of them use a model-free way of performing offloading. The common approaches include state-action-reward-state-action (SARSA), Q-learning, DQN, Actor-critic, and DDPG. This session will cover much of the most relevant research.

- 1) **State-Action-Reward-State-Action (SARSA):** SARSA stands for State-Action-Reward-State-Action. SARSA can be described as an on-policy reinforcement learning methodology that computes continuously updated Q-values based on the agent's interactions with the environment so that it may learn the optimal policy. Five primary components define Q-value: the states, actions, incentives, and the policy. While actions define what the agent can do, states define the current situation of the system. Actions result in the agent being rewarded while also providing feedback for the learning to steer itself in the right direction. Q-value act as an estimator of the expected cumulative reward raised by measures from a particular state, but the guideline policy also serve as a mapping from states to actions that define the

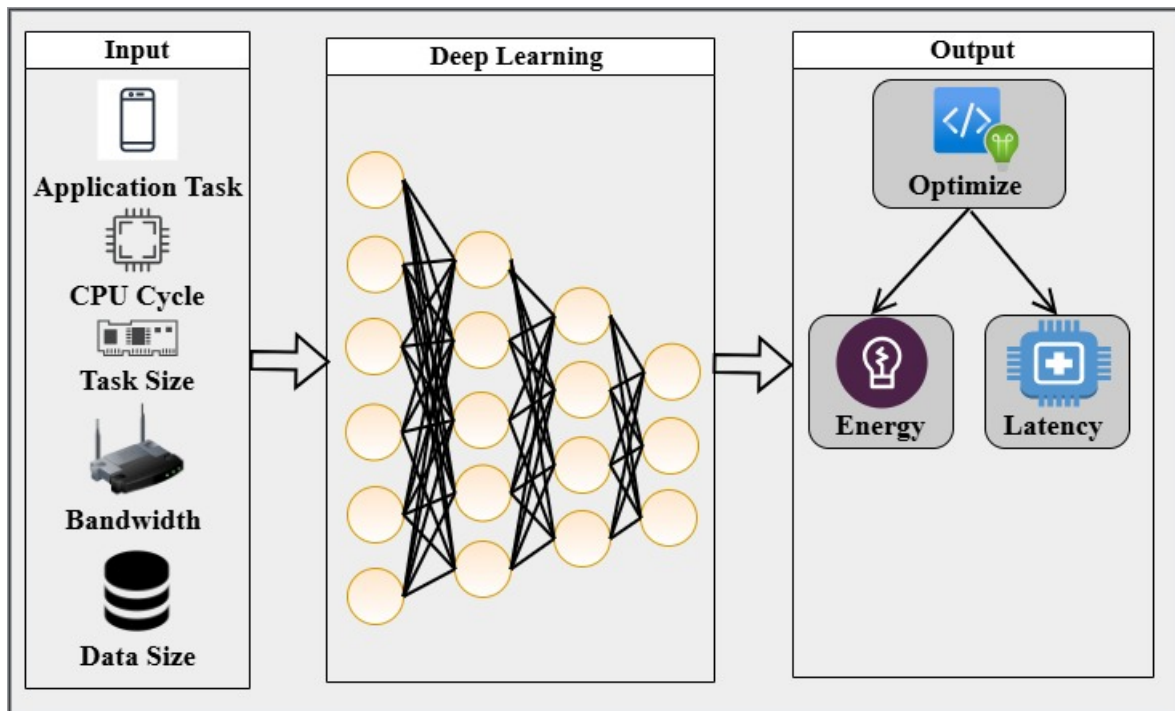


FIGURE 17. Deep learning for optimized task offloading in computation offloading.

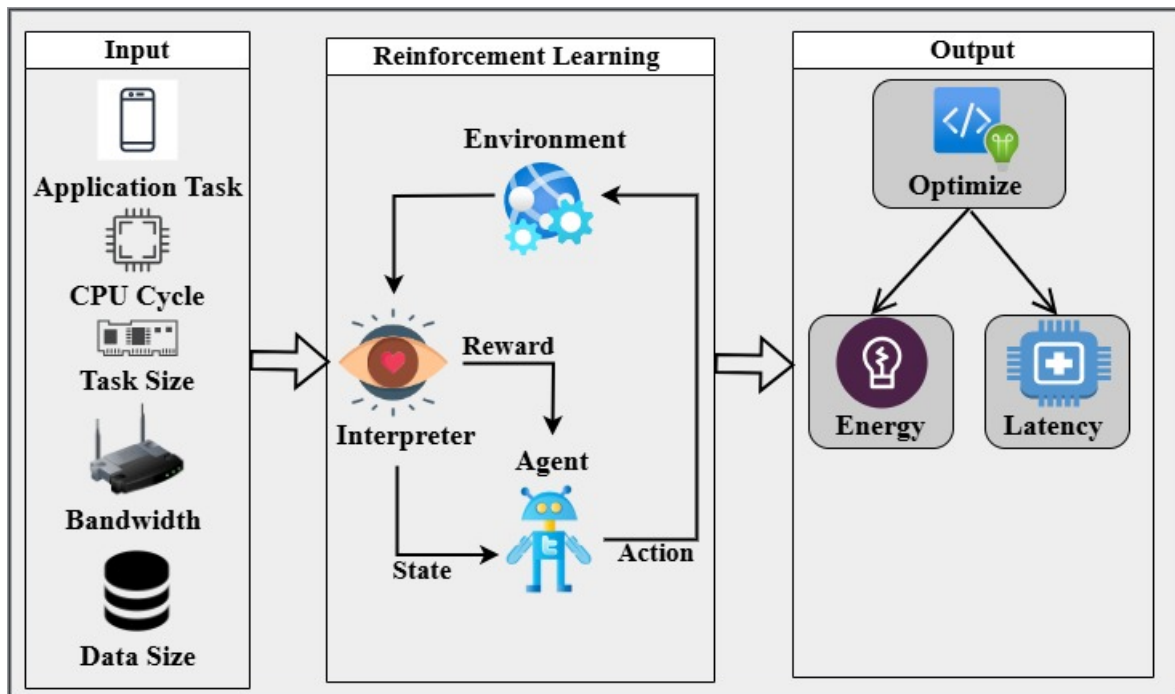


FIGURE 18. Reinforcement learning for adaptive task offloading in computation offloading.

TABLE 12. Comparison of deep learning approaches in computation offloading.

Ref	Problem	Model	Parameters Considered for Decision Making	Result	Scope of improvement
[42]	Optimizing computation offloading in MEC using deep learning for cost efficiency.	Deep Supervised Learning algorithm (DSL) Model	Energy and Latency	Reduced energy consumption, optimized offloading, improved scheduling, and minimized application completion time.	Enhanced offloading decisions, resource allocation, scalability, accuracy, and efficiency using deep learning.
[74]	Efficient DNN offloading for IoT in MEC, optimizing adaptability and execution time.	Deep Neural Network Offloading Model (DNN Off)	DNN-based parameters like initial layer and End layer.	Reduced response time, optimized offloading, high accuracy, minimal overhead, near-optimal performance.	Enhanced adaptability, execution efficiency, dynamic offloading, latency reduction, and resource optimization in MEC.
[53]	Efficient DNN task partitioning and offloading to minimize processing delay in MEC.	Multi-integer linear process-based DNN task offloading	Parameters are derived from the characteristics of DNN layers and their associated tasks.	Achieved up to 90.5% delay reduction using optimized DNN task offloading.	Optimized task partitioning, reduced delay, better resource allocation, and enhanced MEC efficiency.
[75]	Optimizing computation offloading using deep learning for energy efficiency and performance enhancement.	Energy and performance efficient deep learning-based offloading algorithm (EPED)	Energy and Latency	Improved offloading accuracy, reduced energy consumption, enhanced performance, and optimized resource utilization.	Enhanced energy efficiency, optimized task offloading, reduced latency, and improved resource allocation strategies.
[73]	DNN-based object optimization to improve performance on mobile edge devices.	Greedy-based binary offloading decision algorithm to optimize the offloading of DNN-based object detection tasks	Computing power, bandwidth, FPS, delay, offloading decision, latency, SNR, requests, and execution time.	Improved detection performance, reduced latency, optimized offloading, and enhanced resource utilization efficiency.	Enhanced resource utilization, reduced latency, optimized offloading, and improved detection performance in edge computing.
[118]	Enhancing urban management through real-time crowd density monitoring in smart cities	Deep learning-based model ConvLSTM, Red fox optimization, and Gaussian filtering	DNN layers	Achieved 98.40% accuracy in crowd density monitoring model	Enhance crowd density estimation accuracy and real-time performance significantly.
[119]	Optimizing accuracy-resource trade-offs in deep learning model configuration	Deep learning Model like MobileNet V1 and YOLO v3.	Accuracy, resource usage, context variability, and approximation	MPC+MobileNet-V1 achieve 92.9% accuracy with efficient resource consumption	Dynamic adaptation for efficiency and accuracy in diverse applications.

agent's behavior. This enables SARSA to effectively balance exploration and exploitation, making it highly adaptable in dynamic scenarios. To update this Q-value in SARSA, define the following update rules, and it can be calculated using the equation 21, as shown below:

$$Q(X, Y) \leftarrow Q(X, Y) + \alpha [Z + \gamma Q(X', Y') - Q(X, Y)] \quad (21)$$

Where α is the learning rate, γ is the discount factor, Z is the received reward, and $Q(X', Y')$ is the Q-value of the next state-action pair. There are a few update-based studies on SARSA. Taha and Hassan et al. [58] use a reinforcement learning-based technique known as OD-SARSA for cyber-physical-social systems (CPSS) on mobile edge computing networks (MECNs) It shows

that OD-SARSA works better than conventional Q-learning techniques by evaluating offloading options to reduce energy usage and processing delay.

- 2) **Q-learning:** Q-learning, an off-policy reinforcement algorithm, is thus the model that can be used to identify the best action-selection strategy for an agent in a particular environment. To maximize cumulative rewards over time, the agent adopts trial-and-error behavior by trying different actions, learning from the rewards received, and using the information to update knowledge. It is updated with the maximum possible reward from the upcoming state instead of referring to the current policy of the agent. The formula for Q-value updating is given by, and it can be computed using the equation

22, as shown below:

$$Q(X, Y) \leftarrow Q(X, Y) + \alpha \left[Z + \gamma \max_{A'} Q(X', Y') - Q(X, Y) \right] \quad (22)$$

where α is the learning rate, γ is the discount factor, R is the immediate reward, and $\max_{A'} Q(X', Y')$ represents the highest Q-value for the next state S' . We compare eq. 21 and eq. 22, resulting in a finding that Q-learning, in contrast to SARSA, considers each possible action-state pair. The only weakness of this approach is the very low scalability, particularly for very large numbers of state-action pairs. The most important previous studies of Q-learning are listed in Table 13. There are some Q-Learning-based studies focused on minimizing the energy and latency. Some research [76] and [27] proposed a reinforcement learning-based paradigm for improving computation offloading in Internet of Things devices with energy harvesting. DQN-based offloading techniques and hot booting Q-learning address the dynamic nature of MEC networks, allowing IoT devices to make efficient offloading decisions without requiring prior knowledge of MEC models or energy consumption trends. In Q-learning, simulation verifies the suggested approaches by demonstrating decreased energy usage, decreased computation delay, and minimized task drop ratios. In contrast to Q-learning, policy gradient methods optimize the policy function $\pi_\theta(a | s)$ directly, thus are better suited for continuous action spaces and dynamic environments. Q-learning depends on learning discrete Q-values and updating them, whereas policy gradient methods involve adjusting the parameters of the policy in order to maximize the expected reward. While they require more samples and tend to have higher variance, policy gradient methods respond quicker to changes in network conditions. Integrating the two methods can offer better offloading decisions, and it balances the sample efficiency of Q-learning with the flexibility of policy gradients, optimizing the energy-latency balance in MEC systems.

- 3) **Deep Q-Network:** The Deep Q-network, or DQN, is an advanced reinforcement learning approach that generalizes Q-learning by approximating the Q-value function with deep neural networks. For the case of DQN, the decision-making-generating system solves the computation offloading task based on system load, energy constraints, and real-time network conditions regarding whether a task is to be executed locally or offloaded to an edge/cloud server. The agent (mobile device) interacts with the environment (MEC system) by performing local execution and task offloading. By evaluating parameters such as task size, CPU cycles, available bandwidth, and latency restrictions, the DQN model predicts the best offloading decision. The DQN

Q-value updating rule is:

$$Q(X, Y; \theta) = r + \gamma \max_{A'} Q(X', Y'; \theta') \quad (23)$$

This equation 23 shows the Bellman equation for evaluating the Q-function in Reinforcement learning, which is frequently used in guiding DQNs (Deep Q-Networks). Where θ represents the deep neural network parameters, α is the learning rate, γ is the discount factor, and R is the reward. This issue can be tackled with Deep Q Network (DQN) [27]. The DQN uses the convolutional neural network (CNN) to form the value function vector. The main drawback of DQNs is that they are prone to overfitting [77].

- 4) **Deep Reinforcement Learning (DRL):** There has been an exploration of many methodologies so far for implementing DRL in offloading. The Policy Gradient (PG) [29] in this case is one attempt to evaluate the DRL behaviors as it is on-policy and takes a state as input and evaluates action probabilities; it is reasonably convergent [78]. The goal is to set the optimal policy parameter θ in NN. The gradient $\nabla_\theta J(\theta)$ in the Policy Gradient (PG) method is computed using the equation 24, as shown below:

$$\nabla_\theta J(\theta) = E_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \nabla_\theta \log \pi_\theta(A_t | S_t) R_t \right] \quad (24)$$

where θ represents the policy parameters, $\pi_\theta(A_t | S_t)$ is the probability of taking action A_t given state S_t , parameterized by θ , R_t is the cumulative reward at time step t , and τ represent a trajectory sampled from the policy π_θ . This equation 24 was derived for updating the policy parameter and is, therefore, defined as the expectation of the gradient of the log-probabilities of the policy with respect to the reward. Hao and Chao et al. [78] proposed a novel deep reinforcement learning (DRL)-based task offloading algorithm to handle the task offloading problem in single-user Mobile Edge computing(MEC) systems. The algorithm's main objective is to reduce the average energy consumption and average latency for lag procedures on mobile devices. The system successfully learns a roughly ideal offloading approach that accomplishes these goals through training.

The Deep Reinforcement Learning-based Online Offloading (DROO) method for computing offloading choices in wirelessly powered MEC networks is one such DRL-based solution. Apart from that, DROO ensures near-optimal performance, obviating the need for tackling mixed-integer programming issues and reducing computing complexity to a great extent [79]. With simulation results, it is stated that DROO saves a ton of time at around 99.5% processing rate of near-optimal benchmarks. Application-specific DDPG-based solutions have also been widely studied in the context of multi-server, multi-task MEC scenarios to increase server utilization with ultra-low latencies via area-specific parallel

work offloading [80]. Formulated in terms of an MDP, the task offloading techniques optimize long-term performance and server efficiency. Furthermore, they have been used in adaptive compute offloading in multi-access edge computing (MEC) environments, where deep reinforcement learning DRL was combined with deep neural networks to demonstrate improvement in resource usage, energy efficiency, and reduced latencies [35]. Reinforcement learning has surprisingly improved computation offloading techniques in MEC and IoT contexts, as it contributes to making decisions regarding dynamic resource allocation, energy efficiency, and latency reduction. Techniques in this genre include SARSA, Q-learning, and deep reinforcement learning, such as DQN, DDPG, and DROO. Future research should be directed toward scalability for application in real-world deployment and integration with advanced technologies such as 6G and federated learning to further increase the efficiency of use in offloading distributed edge computing environments.

6) Federated Learning

Federated learning (FL) has emerged as a viable alternative to consider in the decentralization of training for any model in contexts where data privacy, energy conservation, and communication overhead are critical concerns. Across the reviewed paper in Table 15, FL is applied to dynamic environments like UAV-assisted vehicular edge computing (VEC), semantic-aware edge network, 5G-enabled IoT, and mobile cloud infrastructure - each introducing tailored innovations to address FL's limitations in real-world applications. All the reviewed studies [81] - [86] validate their proposed federated learning framework through extensive simulations, using tools such as MATLAB, Python-based environments, or custom-designed federated simulations. These simulations reflect dynamic real-world conditions, including vehicular mobility, heterogeneous edge nodes, and varying network loads. Chunlin Li and Chaoyue Deng et al. [81], simulations depict UAV-assisted vehicular edge computing scenarios where GFL-PEARL makes decisions under energy-delay tradeoffs. There is evidence that simulation results have led to a 31% reduction in average system cost and lowered task timeout rates by 56%. Avi Deb Raha and Apurba Adhikary et al. [82] focus on the fl system, entropy-driven semantic offloading, server resource allocation management, with tests conducted for both IID and non-IID data distribution. The simulation presents an increase in accuracy of nearly 5.7% and bandwidth saving of around 86%, proving that semantic confidence metrics can be used for intelligent offloading. In computation offloading, as shown in Figure 19, Devices transfer computation tasks to nearby edge servers, which train local models using decentralized data. These local model updates are transmitted to the cloud node, which combines them to create a global model. The updated global model is then sent back to the edge servers. This process allows for privacy-protected, distributed learning while maximizing resource efficiency and minimizing latency in intelligent networks. FL is used to enable intelligent decision-

making about when and where to offload processing tasks (e.g., to a cloud, an edge server, a UAV, or another peer device), considering dynamic parameters based on energy, latency, bandwidth, and user context.

Mathematical Formulation of FL in Computation Offloading:

Consider a federated learning setup with N clients. Each client i holds a private local dataset $\mathcal{D}_i$. The objective is to collaboratively learn a global model $w \in \mathbb{R}^d$ that minimizes the aggregated empirical risk across all clients.

- **Global Optimization Objective:** The federated optimization problem can be expressed in the equation 25:

$$\min_w \sum_{i=1}^N p_i \cdot L_i(w) \quad (25)$$

where, $L_i(w)$ is the local loss function at client i (e.g., cross-entropy, mean squared error), $p_i = \frac{|\mathcal{D}_i|}{\sum_{j=1}^N |\mathcal{D}_j|}$ is the relative weight of client i based on dataset size. This ensures that clients with larger datasets contribute more significantly to the global model update.

- **Local Model Update:** Each client performs a local update using Stochastic Gradient Descent (SGD) or a similar optimization method. For client i , the model update at communication round t is given in the equation 26:

$$w_i^{(t+1)} = w^{(t)} - \eta \cdot \nabla L_i(w^{(t)}) \quad (26)$$

where, $w^{(t)}$ is the global model received from the server at round t , η is the learning rate, $\nabla L_i(w^{(t)})$ is the gradient of the local loss function at client i with respect to the model parameters. This update is typically performed over a few local epochs before communicating the result back to the central server for aggregation.

- **Aggregation at the Server (FedAvg):** Once local updates are completed, the central server performs a weighted aggregation of the updated local models from all participating clients. This aggregation forms the updated global model using the Federated Averaging (FedAvg) algorithm, defined in equation 27:

$$w^{(t+1)} = \sum_{i=1}^N p_i \cdot w_i^{(t+1)} \quad (27)$$

where, $w_i^{(t+1)}$ is the updated local model from client i at round $t + 1$, $p_i = \frac{|\mathcal{D}_i|}{\sum_{j=1}^N |\mathcal{D}_j|}$ is the relative weight of client i based on its local dataset size. This aggregation step completes one round of federated training. The process is repeated over multiple communication rounds until convergence is achieved.

Federated Learning is a very promising tool for the privacy-preserving, adaptive, and resource-efficient computation offloading over distributed mobile networks. The machine learning models present a good framework for collaborative optimization, while real-time simulations (see references [81] - [86]) attest to significant performance improvements

TABLE 13. Reinforcement learning based offloading approaches.

Ref	Problem	Model	Parameters Considered for Decision Making	Result	Scope of improvement
[58]	Enhancing task offloading and resource allocation in MEC through the application of SARSA reinforcement learning.	RL-SARSA (Reinforcement learning-based State-Action-Reward-State-Action) algorithm	Task size, bandwidth, CPU frequency, energy consumption, latency, offloading decision, processing time.	Increased offloading effectiveness, decreased energy usage, decreased latency, and SARSA-optimized resource allocation.	Improved decision-making, efficient resource allocation, minimized latency, and increased energy efficiency.
[59]	Enhancing caching, offloading, and resource allocation in dynamic MEC through the application of deep reinforcement learning.	Deep Deterministic Policy Gradient (DDPG) algorithm	Task size, bandwidth, cache status, energy consumption, latency, offloading decision, resource allocation.	It improved MEC systems performance using DRL; it minimized energy consumption and increased offloading efficiency.	Enhanced caching, offloading, resource allocation, energy efficiency, latency reduction, and MEC performance optimization.
[29]	Enhancing the efficiency of decentralized computation offloading in multi-user mobile edge computing through the application of deep reinforcement learning.	Deep reinforcement learning (DRL) based dynamic computation offloading strategy	Task size, power allocation, bandwidth, latency, energy consumption, queue length, SINR, offloading decision.	Enhanced multi-user MEC performance, decreased power consumption, decreased latency, and improved offloading efficiency.	Enhanced offloading decisions, power efficiency, reduced latency, scalability, and optimized multi-user MEC performance.
[106]	Optimizing offloading and auto-scaling in energy-harvesting MEC using online reinforcement learning.	Post-Decision State (PDS)-based reinforcement learning algorithm	Workload arrival, energy harvesting, battery state, latency, power demand, offloading, auto-scaling decisions.	Improved MEC system performance, lower latency, cheaper energy costs, and better offloading choices.	Enhanced learning speed, optimized offloading, energy efficiency, reduced latency, and improved resource management.
[76]	Optimizing IoT computation offloading with energy harvesting using reinforcement learning techniques.	Transfer Learning-based Q-learning Method	Battery level, energy harvesting rate, bandwidth, task size, latency, offloading decision, computation cost.	Reduced energy consumption, decreased latency, enhanced offloading efficiency, and optimized resource use in IoT.	Enhanced learning speed, optimized offloading, energy efficiency, reduced latency, and improved resource allocation.
[78]	Enhancing task offloading in MEC through deep reinforcement learning to improve efficiency.	Deep reinforcement learning-based offloading	Task size, energy consumption, latency, bandwidth, queue length, CPU usage, offloading decision.	It reduces the average energy usage of mobile devices and the average delay experienced by tasks waiting in the queue	it enhances scheduling, lowers energy consumption, reduces latency, and optimizes performance.
[79]	Enhancing online computation offloading in wireless-powered MEC through the application of deep reinforcement learning.	Deep reinforcement learning-based online offloading (DROO) framework	Energy harvesting rate, channel gain, task size, latency, bandwidth, power allocation, offloading decision.	Energy consumption is minimized, latency and offloading efficiency are increased, and resource allocation is optimized through DRL.	Enhanced offloading decisions, energy efficiency, reduced latency, adaptive learning, and optimized resource allocation.
[80]	Optimizing multi-task offloading in MEC for ultra-low latency and efficient resource utilization.	Deep Deterministic Policy Gradient (DDPG)-based task offloading approach	Task size, latency, server load, bandwidth, mobility, energy consumption, offloading decision.	Improved system performance, reduced latency, higher utilization of servers, and better offloading efficiency.	Enhanced server utilization, reduced latency, optimized offloading, improved mobility handling, and resource efficiency.

TABLE 14. (Continued) Reinforcement learning based offloading approaches.

Ref	Problem	Model	Parameters Considered for Decision Making	Result	Scope of improvement
[35]	Enhancing computation offloading in MEC through machine learning techniques to improve efficiency and performance.	Deep Deterministic Policy Gradient (DDPG)-based offloading using MDP and Deep Neural Network (DNN) for prediction and decision making	Task size, bandwidth, latency, energy consumption, computing power, offloading decision, resource allocation.	Improved offloading efficiency, reduced latency, optimized resource allocation, and enhanced MEC system performance.	Enhanced ML models, improved offloading decisions, reduced latency, energy efficiency, and resource optimization.
[27]	Enhancing the efficiency and performance of edge computing through the optimization of RL-based computation offloading	Q-learning and Deep Q-network (DQN)-based algorithm	Task size, bandwidth, latency, energy consumption, computation cost, offloading decision, resource allocation.	Improved offloading efficiency, reduced latency, optimized resource allocation, and enhanced edge computing performance.	Enhanced RL algorithms, improved offloading decisions, reduced latency, energy efficiency, and resource optimization.
[47]	Minimize total completion time of computational tasks in ARMS	Deep reinforcement learning	CPU cycle, GPU, and memory utilization rates	Improve tasks completion times and resource utilization in ARMS	Enhancements in computation offloading, resource use, and task efficiency.
[120]	Minimize delay and energy consumption in vehicular task processing.	Deep deterministic policy gradient (DDPG) algorithm	Learning rates, mini-batch size, and experience replay buffer size.	The proposed scheme reduces latency and energy consumption significantly.	Future improvement focuses on overhead analysis and framework enhancement.
[121]	Optimize IoT task offloading to minimize latency and energy consumption.	Gale-Shapley algorithm	Input size, output size, computational demand, deadline, and transmission power.	Proposed a strategy to reduce energy consumption and enhance latency performance.	Improvement focuses on enhancing scalability in IoT networks.
[122]	Balancing user and provider interests in edge computing task offloading	Twin delayed deep deterministic policy gradient (TD3)	Task arrival rates, energy consumption, pricing, and offloading decision.	Dynamic pricing enhances task allocation in edge computing networks.	It optimizes task offloading, balancing user and provider benefits effectively.
[123]	Efficiently schedule IoE tasks, minimize time, reduce energy, and ensure security.	Reinforcement learning	Processing capabilities, energy profiles, data volume, priorities, security level, and workloads	Improved efficiency and security, with reduced energy consumption	Need to improve the scalability, real-world testing, hybrid models, lightweight security, QoS frameworks, and privacy.

in latency, energy consumption, model accuracy, and bandwidth savings. Some more literature survey shows that Milad Rahmati et al. [85] simulate a 5G-enabled IoT network with an energy-constrained, adversarial setting, evaluating how the EAFL model—with ACS, GAT, and blockchain integration, manages model accuracy (91.8%) while reducing energy usage by 35.4% and lowering attack success rates by 72.3%. Meanwhile, Nipun Agarwal and Sandeep Joshi et al. [86] model a mobile edge environment for UAVs, incorporating dfedavg within a Markov decision process. Simulation outputs confirm up to 35% improvement in energy efficiency and up to 16.7% latency reduction over baseline schemes like DDPG and centralized offloading. Across the reviewed studies, FL strategies vary significantly in their handling of communication cost, convergence speed, and privacy-efficiency trade-offs. For instance, semantic-aware FL [82] achieves substantial bandwidth saving (~86%) through se-

lective feature transmission but at the expense of increased local computation. UAV-assisted GFL-PEARL [81] balances energy-delay trade-offs but requires more communication rounds for convergence in high-mobility environments. Approaches integrating blockchain or graph attention networks [85] enhance privacy and robustness but introduce additional processing overhead, slightly impacting convergence time. Overall, these studies confirm that FL is evolving beyond basic privacy preservation into a multi-faceted enabler for adaptive, secure, and energy-aware computation across distribution networks. Future works should include the tight integration of FL with online scheduling, semantic communication, and cross-layer optimization, with a view toward envisioning real-world deployments.

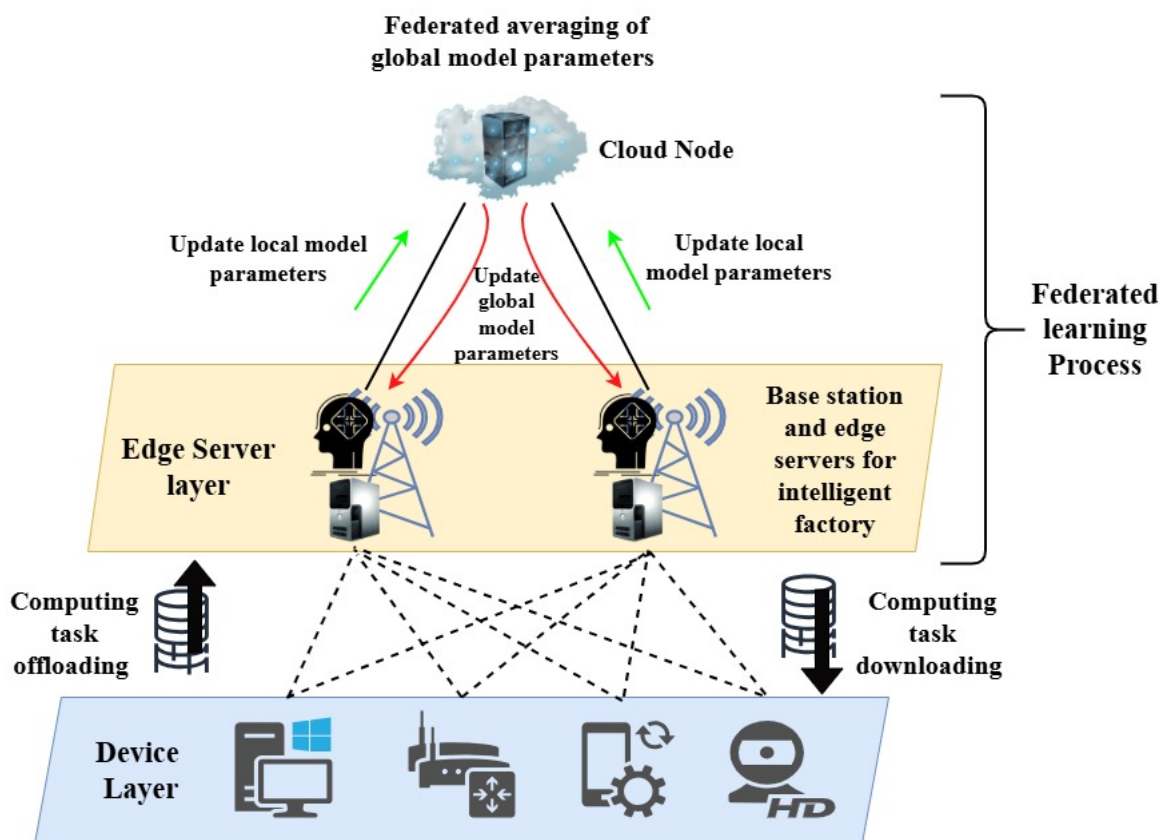


FIGURE 19. A comprehensive overview of the federated learning process, which involves the transfer of data and computation across various layers, including devices, edges, and clouds, to enable intelligent computation offloading.

7) Blockchain-based offloading:

Blockchain technology was developed to secure cryptocurrency transactions, the most significant application of this technology. Nowadays, blockchain technology acts as a fundamental tool for decentralized computing. In the offloading paradigm, which tasks are shifted away from resource-constrained devices (at times called smartphones, vehicles, or sensors) to more powerful nodes (such as edge servers, UAVs, or cloud), blockchain is involved in maintaining a secure, transparent, and tamper-proof ledger regarding task execution, payments, and authentication of documents. As shown in Figure 20, a complex architecture combining IoT, fog, cloud, and blockchain networks to ensure secure and efficient data processing. IoT devices transmit task data to nearby fog nodes, which then process or forward tasks to cloud nodes. the results of the task are delivered in the same manner. Blockchain guarantees secure data exchange and validation across all layers, with peers recording and verifying transactions. This architectural design strengthens trust, minimizes latency, and evenly distributes computational tasks while preserving data integrity in IoT-driven environments. Traditional offloading mechanisms usually have a centralized coordinator that leads to a single point of failure, trust issues, and bottlenecks in terms of scalability. These issues are all solved by blockchain, which removes central-

ized trusts by having offloading decisions and exchanges of resources recorded and validated in a cryptographically secure manner. Recent research studies in table 16, [87] - [91] have shown that blockchain serves as a decentralized, tamper-resistant layer that ensures secure communication, verifiable transactions, and transparent task management. Matheen Fathima G and Shakkeera L et al. [87] blockchain is combined with federated learning to improve the security, traceability, and audibility of local model updates in mobile cloud computing (MCC). By utilizing Hyperledger Fabric and Ethereum-based smart contracts, this framework minimizes system overhead by more than 20% while ensuring privacy-preserving coordination of models. Similarly, Manuel J. and C. S. Reis et al. [88] address the security vulnerabilities present in 5 G-enabled IoT networks. In this context, blockchain serves as the foundation of an edge-authentication system, effectively tackling challenges like distribution denial-of-service (DDoS) attacks and unauthorized data breaches. The system utilizes cryptographic hashing and layered blockchain to minimize latency and achieve a success rate of over 52% even in the face of adversarial conditions. Zhaolong Gao and Wei Yan et al. [89] blockchain technology is combined with a hybrid deep learning model (VCD-TSNet) to enable real-time, secure data processing in large-scale IoT systems. The implementation of blockchain

TABLE 15. Federated Learning based offloading approaches.

Ref	Problem	Model	Parameters Considered for Decision Making	Result	Scope of improvement
[81]	Traditional federated learning fails to generalize in the dynamic context of UAV-enabled VEC; hence, there is a need to optimize computation offloading, considering latency and energy consumption.	Federated learning (FML)+ GFL-PEARL	Energy- delay tradeoff, task size, UAV location, DAG context	Reduced cost and time-out, faster convergence, high accuracy, outperformed benchmark in dynamic VEC environment.	Improve generalization, real-world deployment, scalability, fault tolerance, and integration with diverse vehicular IoT platforms.
[82]	FL at edge faces high communication cost, latency, privacy issues, and a single point of failure.	FL with ConvNeXt-B/V2 and semantic offloading.	Entropy, bandwidth, delay, offloading threshold, model assignment	↑ Accuracy (2.6–5.7%), ↓ Bandwidth (58–87%)	Apply to real-world use, distillation, resilience, optimize thresholds
[83]	High task load, limited vehicle resources in IoV; need for fair, efficient offloading.	FL-CEGID (Federated learning with cloud-edge game + Incentive)	Task priority, reward, bandwidth, offloading cost, RSU model accuracy/loss.	↑ Throughput, revenue, satisfaction; ↓ latency and resource use; better fairness	Improve resource use under heavy traffic; optimize incentives; lower latency in large networks.
[84]	Centralized offloading is slow and unsuitable for dynamic, large-scale VEC systems.	MCM-FDRL (Multi-agent collaboration with federated deep reinforcement learning)	Local load, transmission rate, vehicle density, response time, edge resources	↓ Avg response time by 9.75-64.9%; ↓ exec. time by 99%; fast convergence	Handle extreme-scale mobility, further reduce delay, optimize federated updates
[85]	FL in 5G-IoT faces high energy use, poor resource allocation, and security threats.	EAFL (Energy-Aware FL with ACS, QAT, BES)	Energy, compute power, bandwidth, attack risk, gradient size.	↓ Energy 35.4%, ↓ Attack 72.3%, ↑ Accuracy 91.8%, ↓ Communication 40%	Lighter blockchain, real-world test, adaptive compression, hybrid FL, privacy enhancements
[86]	UAV-based MEC offloading faces delay, energy, and scheduling challenges in dynamic networks	DFedAvg (Decentralized federated averaging)	Offload ratio, UAV coords/speed, task size, UE power, CPU freq, delay, energy	↓ Delay up to 16.7%, ↑ Energy efficiency by 35%, better scheduling and mobility	Multi-UAV setup, data layer for bandwidth, support larger- scale mobile UEs.

guarantees data immutability and audibility, enabling a high throughput (920 TPS) and a low mean absolute error (0.28), while also ensuring quantum-resilient scalability.

Research [87] - [91] has identified four core functionalities of blockchain in this context:

- Task verified and record keeping, where smart contracts log task metadata, execution result, and timestamps on-chain,
- Access control and identity management, which restricts task participation to authenticated entities using digital signatures and public keys,
- Decentralized micropayment systems, which allow automatic reward distribution through gas-based or tokenized compensation, and
- Consensus-driven resource allocation, which guarantees fair scheduling and prevents manipulation of offloading tasks.

Mathematical Formulations in Blockchain-Based Offloading:

- Task Offloading Model with Utility Function:

Blockchain-integrated offloading models frequently utilize a utility function that optimizes task assignment by considering various factors. A function that can measure the overall satisfaction of a person. A generalized utility function can be defined in the equation 28:

$$U_i = \alpha D_i + \beta E_i + \gamma C_i - \delta S_i \quad (28)$$

where, U_i is the overall utility score for task i , D_i is the total delay, E_i is the energy consumption, C_i represents blockchain-related cost (e.g., gas fees), and S_i denotes the secrecy rate or security level. The coefficients α , β , γ , and δ are system-defined weights that reflect the relative importance of each factor according to policy preferences or environmental constraints.

- Blockchain Consensus Delay in Computation Offloading: Let T_c represent the total time required to reach consensus during a task offloading transaction in a blockchain-enabled system. This can be defined in the equation 29:

$$T_c = T_p + T_b \quad (29)$$

where, T_p is the time taken to propagate the block to all participating nodes, T_b is the time required for block validation, which varies based on the consensus mechanism used (e.g., Proof of Work (PoW), Practical Byzantine Fault Tolerance (PBFT), etc.). In lightweight blockchain environments, particularly those integrated with mobile edge computing and IoT systems, PBFT is commonly adopted due to its low-latency and energy-efficient characteristics (see Refs [88], [90]).

- **Smart Contract Execution Cost:** In a blockchain system similar to Ethereum, the cost of executing a smart contract is measured in terms of cost. This cost is calculated as given in equation 30:

$$C_{\text{gas}} = \sum_{i=1}^n G_i \cdot P \quad (30)$$

where, G_i represents the gas units required for the i -th operation within the smart contract, P denotes the current gas price (typically measured in Gwei or another cryptocurrency unit). This total gas cost C_{gas} can be incorporated into the overall utility function U_i for computation offloading, allowing the system to make more *cost-aware decisions* during task scheduling and resource allocation.

Use cases encompass trustless collaboration (eliminating central schedulers), immutable logging (for audit trails), and secure payment systems (via smart contracts). For example, Refs [87] – [91] illustrate how blockchain technology can facilitate energy-efficient federated learning, ensure data integrity in the internet of things (IoT), enable secure offloading of unmanned aerial vehicles (UAVs), and enable decentralized trading of resources. Some more literature survey like, Feng Xu and Zitong Zhao et al. suggests a flexible framework that integrates blockchain technology with Deep Reinforcement Learning (TD3) in mobile edge computing. The blockchain layer guarantees trust and agreement among edge nodes, facilitating dependable task scheduling and equitable offloading in a network divided into smaller parts. This method leads to significant improvements in latency reduction and resource utilization, making it suitable for an environment with multiple concurrent tasks. Lastly, Ting Chen and Shujiao Wang et al. [91] discuss the computational and communication expenses in blockchain-enabled V2X networks by utilizing IRS-equipped UAVs. Blockchain technology is employed in this scenario to handle gas fees, monitor offloading decisions, and guarantee data privacy through the use of smart contracts. The outcome is a system that not only enhances the efficiency and effectiveness of tasks but also greatly improves the level of secrecy in mobile, challenging environments. Overall, blockchain technology is a powerful tool that allows for secure and trustworthy computation to be offloaded. By securely recording every offloading transaction, allowing autonomous payment through

smart contracts, and ensuring privacy through cryptographic techniques, blockchain enhances the capabilities of AI-driven and FL-based offloading frameworks. Research in this space (Refs [87] – [91]) highlights its increasing influence in mobile cloud, IoT, UAV, and vehicular edge computing, resulting in noticeable enhancement in latency, energy consumption, and system protection. This Comparative analysis Table 17 highlights the various ways that machine learning and modern techniques address the fundamental problem of computing offloading. Supervised learning methods work well for static and predictable workloads and enable devices to offload or execute locally with high precision when sufficient labeled data is available [92]. In contrast, unsupervised learning methods, which identify clusters of analogous workload or data types, support enhanced resource sharing and scheduling [93]. While these methods are useful, they offer no solution to the issues of responding to latency variations in resource availability. The real-time edge computing scenario is best served by offloading and energy-aware heuristics, such as Q-learning and deep Q-network [94], [95] and deep reinforcement learning. Such a method can optimize the offloading decision and adapt it to rapidly changing network conditions and energy supply. Model like federated learning increases this flexibility even further by protecting user data privacy and enabling rapid, decentralized decision making by training models on the edge devices themselves [96], [97]. As deep learning techniques are able to uncover deeply nested relationships within workload and network data, they allow for predictive resource allocation and latency-aware optimization, which must be trained with a large amount of data for proper functioning [98] [99]. In the end, blockchain-based offloading methods improve the security and trust aspect of offloading by providing decentralization and a tamper-proof system for task allocation, payment, settlement, and auditing, with a small latency overhead because of the consensus algorithms [94] [100] [101]. As a whole, these methods provide all the necessary tools to allow a computation offloading system to effectively balance latency, energy efficiency, privacy, and security in today's MEC and IoT systems.

VII. AN OVERVIEW OF COMPUTATION OFFLOADING:

A. A BOARD CATEGORIZATION OF COMPUTATION OFFLOADING

Computation offloading can usually be divided into two types: binary computation offloading and partial computation offloading. For category-based review papers on computation offloading, as shown in Table 18. These types depend on the degree of relocation of computational tasks off the remote server. They help optimize resource consumption, accelerate processing efficiency, and improve application performance. Especially in resource-related environments such as mobile computing and edge computing.

- 1) Binary computation offloading
- 2) Partial computation offloading

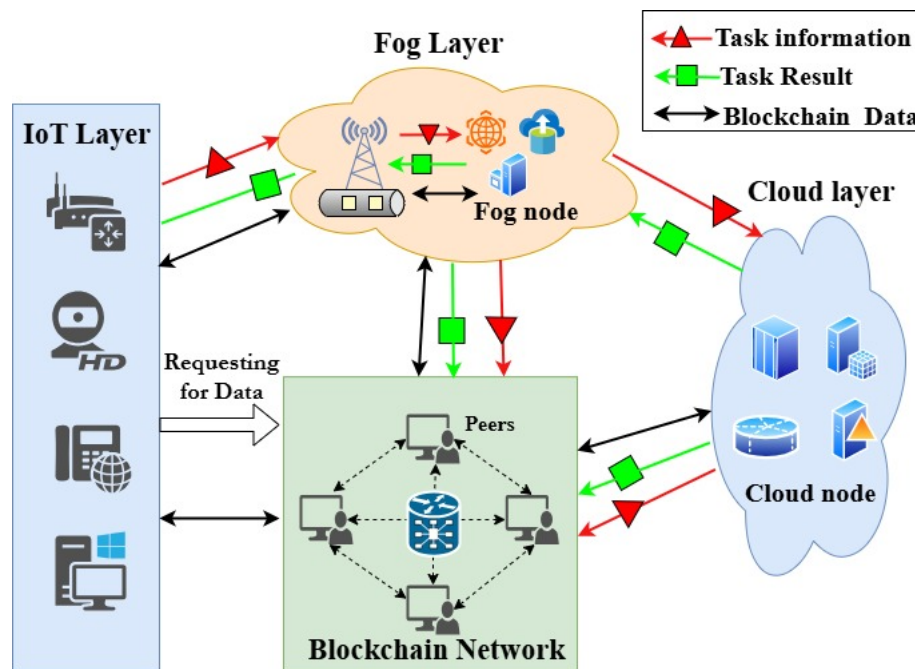


FIGURE 20. A fog-cloud architecture that utilizes blockchain technology for secure and decentralized processing of IoT tasks and data sharing.

TABLE 16. Blockchain-based offloading approaches.

Ref	Problem	Model	Parameters Considered for Decision Making	Result	Scope of improvement
[87]	Inefficient offloading and insecure data sharing in mobile cloud computing	FL + Blockchain + Adaptive salp swarm algorithm (ASSA)	Task deadline, workload, bandwidth, CPU	↓ Overhead (20.14%), ↓ Boot time (20.47ms), ↓ CPU usage (0.45%), ↓ Task failure (2.52%)	Enhance blockchain efficiency; adaptive resource management for heavy loads; optimize FL scalability
[88]	Security issues in 5G-IoT (unauthorized access, DDoS, data tampering)	Hybrid blockchain + AI (CNN, LSTM)	Latency, anomaly score, Access log	Demonstrated resilience under adversarial scenarios using synthetic data. Layer-2 and smart contracts improved performance. Success rate =52.8%	Use ZKPs for privacy, Federated learning for decentralized training, Lightweight AI for edge use.
[89]	Centralized methods fail real-timeIoT needs and security	VCD-TSNet (VGG + ConvLSTM + DNN + Blockchain)	Accuracy, Latency, MAE	↑ Accuracy (97.5%), ↓ MAE (0.28), ↓ Latency (85ms)	Focus on lightweight model design, privacy, and post-quantum secure blockchain scalability for constrained edge device
[90]	Blockchain-MEC integration causes extra delay and overhead; need scalability and real-time decisioning	Scalable + Blockchain + MEC + DRL(TD3)	Task size, delay, frequency, queue, offload policy, consensus time	↑ Task success, ↓ Latency, scalable, low consensus overhead	Improve authentication, resource trading, real-world deployment
[91]	Traditional schemes optimize latency or energy, ignoring total costs (Gas + MEC) and security in presence of eavesdroppers	UAV + IRS + Blockchain + MEC	Delay, Energy, Secrecy rate, Gas cost, CPU, Tx power	Simulation show that joint optimization improves system utility and task success rates significantly	System currently uses fixed gas fee. future work includes adaptive pricing, dynamic trajectory optimization for UAVs, and improved scalability.

TABLE 17. Comparative Table – ML & Emerging Approaches vs. Offloading Challenges

Approach	Key focus	Address task granularity	Latency sensitivity handling	Data locality and privacy	Typical use case in offloading
Supervised Learning [92]	Learns from labeled datasets to predict optimal offloading decision	Good for static and predictable workload (e.g., fixed task size).	Limited– struggles with sudden network changes.	Centralized data required.	Predicting offload vs. local execution decision boundaries.
Unsupervised Learning [95]	Identifies patterns or clusters in unlabeled data.	Group tasks for batch or cooperative offloading	Indirect latency impact– better clustering improves scheduling efficiency.	Data needs central access	IoT workload clustering, anomaly detection.
Reinforcement Learning (RL) [95] [99]	Learn optimal policy via trial-and-error interaction.	Dynamically adjusts task split/merge.	Excellent– adapts to changing network and energy states.	Requires states during training.	Q-learning/DQN for latency and energy-aware dynamic offloading.
Federated Learning (FL) [93] [96] [97]	Trains models locally, shares only gradients.	Personalized per-device models for task-specific granularity.	Fast decision at the edge, no central dependency.	High– raw data never leaves devices.	Privacy-preserving, collaborative offloading in MEC.
Deep Learning (DL) [92] [99]	Learns complex patterns and nonlinear relationships.	Can infer task size/resource needs from complex input (e.g., image, time-series).	Predicts latency trends and optimizes decisions in advance.	Requires large datasets and training Overheard.	CNN/RNN models for predictive offloading and resource allocation.
Blockchain-based approaches [94] [100] [101]	Ensure secure, tamper-proof task allocation and payment settlement.	Neutral– focuses on trust rather than task size.	Latency slightly higher due to consensus overhead.	Very high– decentralized and immutable transaction logs.	Trustless tasks scheduling, secure offloading agreements between devices and MEC nodes.

1) Binary Computation Offloading

For binary computation offloading, the computation is either completely offloaded to the remote server or fully performed on the local device. This means that it operates according to the "all-or-nothing" decision rule, which includes factors like the network condition, computational complexity, and energy consumption. This makes it suitable for applications requiring a major amount of computational power while being low in latency and energy efficient. However, this may not be a solution applicable in a condition where partial execution may yield better performance.

2) Partial Computation Offloading

Partial computation offloading is that in which only selective portions of tasks out of the total tasks on a particular application or process might be moved to a remote server, while the other sections are still executed locally. This strategy renders the use of available bandwidth and compute resources more efficient. Most of the time partial offloading is preferred where applications or programs have different requirements for processing, for example, real-time data analysis, artificial intelligence workloads, and interactive applications.

VIII. PROBLEM ARCHITECTURE: SYSTEM MODEL

There are several ways of visualizing the offloading problem. One approach is to consider that the user submits the task to

TABLE 18. Classification of offloading categories.

Offloading Category	Reference
Binary computation offloading	[42], [62], [63], [58], [59], [29], [64], [65], [106], [76], [78], [79], [71], [80], [73], [112], [54], [4], [72], [34]
Partial computation offloading	[70], [74], [53], [75], [33], [35], [27]

an aggregator, which further uses its own set of workers to perform the task. The complete architecture [62], [63] can be visualized as a directed acyclic graph (Tree).

In the context of Mobile Edge Computing, the system model can contain the following components:

- 1) Mobile device/edge node/user application/wireless device/user device
- 2) Base station/access point/WLAN AP/small base station/Macro base station
- 3) Mobile Edge System (MES) server
- 4) Mobile edge computing (MEC)/Remote edge server
- 5) Cloud server/cloud data center/Potential association
- 6) High-performance server

A typical scenario consists of mobile devices generating data or capturing data from real-time systems. This data needs to be processed, and this processing results in tasks

to be completed. In real-world deployment, computation offloading architectures in actual field deployment are often kept aligned with industry standards such as ETSI Multi-access edge computing (MEC) and the 3GPP 5G specifications. ETSI MEC standardizes APIs and reference architecture to provide hosting capabilities at the network edge so that processes can be served with low latency and efficient bandwidth, supporting AI-driven offloading in vehicular networks, UAV-assisted monitoring, and IoT edge analytics, among other [102] [103]. 3GPP 5G standards further enhance offloading feasibility by defining features such as network slicing, Ultra-reliable low latency communication (URLLC), and edge integration, all of which directly affect task migration method, resource allocation policies, and QoS guarantees [104] [105]. Incorporating these standards in computation offloading research creates interoperability, scalability, and a smooth transition from academic research prototype to large-scale real-time implementation. The sequence of actions in the context of the different components can be the following:

A. FULL-STACK DATA FLOW (DEVICE-EDGE-CLOUD)

These situations follow a structured progression from the user/device to the edge (mobile edge computing/base station/access point) and finally to the cloud. As shown in Figure 21, devices produce the data, which can be processed locally or transmitted to edge servers to provide quicker responses. Advanced processing and storage are delegated to the cloud for complex tasks, facilitating efficient, scalable, and low-latency computing across various applications.

- 1) Data → Mobile device → base station → MES server → Cloud
- 2) Local computing layer → Mobile edge computing layer → Cloud computing layer
- 3) Mobile Device → local processing → user interface → Cloud server [65]
- 4) Mobile Device → Access point → Small base station → MEC Server → Cloud data center → High performance server [33]
- 5) User device / Mobile Device → Base station with MEC server → Cloud / Core network [35]
- 6) End Device layer → Edge layer → Cloud layer [27]
- 7) Mobile Device → Base station → Internet → Cloud Server [34]

B. DEVICE-LOCAL/EDGE-ONLY PROCESSING (NO CLOUD)

These are primarily focused on local or edge processing, either due to time limitations or energy efficiency requirements. As shown in Figure 22, it demonstrates a two-tier architecture where devices like mobile phones, laptops, smart homes, and industrial sensors directly connect to edge servers for processing. The lack of a cloud layer suggests that all computations and decision-making take place locally at the edge, resulting in quicker response times and reduced latency.

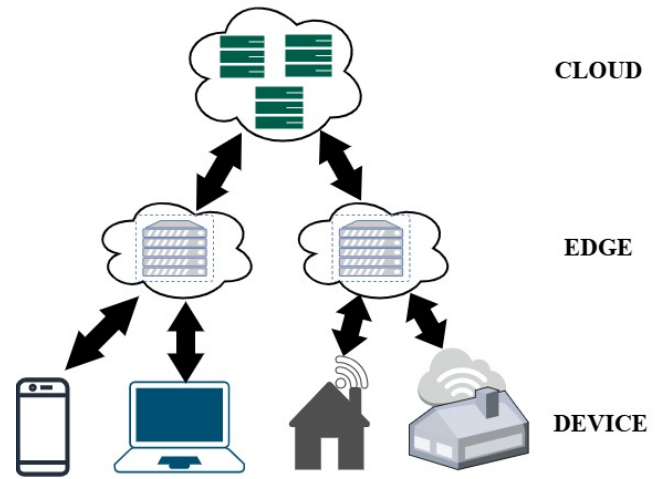


FIGURE 21. Computational offloading in a Device-Edge-Cloud ecosystem.

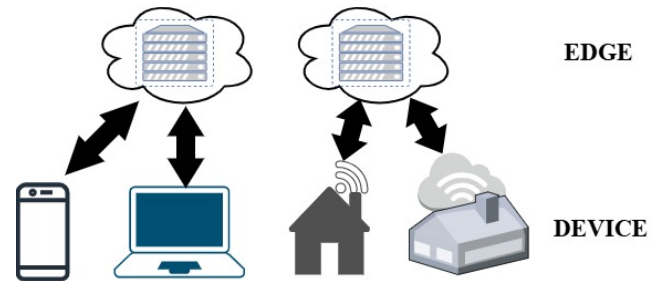


FIGURE 22. Efficient Computing Through Localized Edge Infrastructure

- 1) Energy harvesting → IoT device → MEC server
- 2) Energy harvesting active servers (Local processing) ← Edge Device → Offloading to server connected to power grid [106]
- 3) Mobile device → Small base station → Macro base station [80]
- 4) Mobile Edge device → Wireless LAN AP → Remote server [73]
- 5) Mobile user → Small Base station / macro Base station → MEC Server [4]
- 6) Mobile User → BS with MEC Server → Potential association [56]

IX. COMPUTATION OFFLOADING DATASETS

A. DATASET BASED ANALYSIS

The dataset plays an important role in training, evaluation, and benchmarking for those models in machine learning that relate to their application in particular fields and use cases. As shown in Table 19, a few examples of datasets for use in the machine learning model. "Synthetic dataset" [62] and [42] attend to providing artificial data for the training and evaluation of machine learning. The former focuses on classification, clustering, and predictive modeling, in contrast to the more general approach. The other method is more specific and is typically used for a particular type of task, such as feature engineering, supervised learning, and model verification. Re-

searchers can try different learning strategies in a controlled environment using these artificial datasets before giving the real data the attention it deserves. The next dataset generates its own mathematical model dataset [70] and [73] moving in the direction of reproducing the distribution of real-world data through mathematical means. The beauty of using mathematically generated structured realistic data is their credibility, which can then be used to train machine learning models for very important applications. On the other hand, historical data is very important for better models and making the right decision. The "Historical dataset to train a random forest regression prediction model" [73] is particularly aimed at training random forest regression models, hence is suitable for regression modeling, feature importance evaluation, and benchmarking against other datasets. The next dataset stands out of normal datasets. "GAIA Open dataset (Global Astrometry Interferometer for Astrophysics)" [107] stands for the specialized dataset developed for research. Different from other datasets common to machine learning applications, this dataset has aided the advancement of predictive modeling for astronomical data. It equally holds significance in regression modeling and feature engineering and in different scientific exploits. In all these datasets, in the end, some datasets are constructed to offer controlled, experimental conditions to test algorithm performance.

X. TOOL-BASED ANALYSIS IN COMPUTATION OFFLOADING

The analysis of any computation offloading approach is through tools that involve the usage of different software frameworks, simulators, algorithms, and performance evaluation platforms for assessing and optimizing offloading approaches. As shown in Figure 23, these tools would further benefit researchers and developers as they provide performance measurement for diverse aspects of the offloading process like latency, energy consumption, submit consumption, and various resource allocation.

A. AI AND MACHINE LEARNING-BASED TOOLS:

In the modern world of distributed computing, computation offloading is arguably one of the best tactics used within mobile edge computing (MEC) and cloud settings. AI and machine learning-based tools are, however, used for improving the efficiency and adaptability of decision-making in computation offloading processes. AI-driven tools are used for adaptive decision-making and real-time offloading optimization. As shown in Table 20.

B. SIMULATION AND PERFORMANCE EVALUATION TOOLS

To assess AI- and machine learning-based computation offloading schemes, robust simulation environments and performance evaluation criteria are required. The effectiveness of offloading techniques under dynamic edge computing environments has been assessed with different tools and frameworks, as shown in Table 21. These tools are used

to simulate computation offloading scenarios and evaluate system performance under different conditions.

C. ENERGY AND RESOURCE MANAGEMENT TOOLS

Resource management tools and energy management in computing offloading aim at improving the distribution of computing tasks between local computing devices and remote cloud or edge servers. As shown in Table 22, such tools allow one to reduce energy consumption, make the processing more efficient, and utilize resources better by intelligently allocating workloads.

D. SECURITY AND PRIVACY ANALYSIS TOOLS

Tools for security and privacy analysis in computation offloading devices secure means of data transit, safeguard sensitive information, and offer relief from cyber threats that suddenly emerge as soon as the offloading is done to cloud or edge servers. There are several tools like Wireshark [135], [136], and PyCryptodome [137] for safely offloading your tasks, as shown in Table 23. They recommend real-time monitoring and assessment of risks to unauthorized access or branch of unauthorized data. Computation offloading safety is increased while also ensuring maximum efficiency and data security in distributed environments.

E. HARDWARE-AWARE PERSPECTIVES IN COMPUTATION OFFLOADING

The feasibility and efficiency of computation offloading heavily depend upon the hardware capabilities across devices, edge servers, and cloud infrastructure [138] [139]. Edge servers incorporated GPUs, FPGAs, or AI chips such as Google's edge TPU for high-performance, low-latency processing [138] [140], whereas cloud data centers utilize a large cluster of GPUs and TPUs for massive parallelism, accepting a trade-off for more latency [139]. NPUs are energy-efficient but cannot operate on large-scale tasks. GPUs have higher throughput but are costly in energy [140]. Considering the trade-offs, hardware-aware offloading frameworks can dynamically map tasks to the processing unit that best balances latency, energy consumption, and accuracy, exchanging energy consumption for accuracy from model-level compression [141]. Table 24 summarizes five widely used hardware platforms, covering their description, use case, advantage, strength, and weakness, reflecting the spectrum of hardware solutions from ultra-low-power TinyML accelerators to high-performance cloud GPUs.

XI. CONCLUSION AND FUTURE WORK

In this research study, the results were investigated in various ways, including task generation, computation tasks, task formulation, communication and computation, resource allocation and decision making, challenges in computation offloading, and machine learning models in computation offloading. We reviewed a few research papers on the process of transferring computational tasks to machines in the field of machine learning. We reviewed previous studies

TABLE 19. Dataset based analysis

Reference number	Data set name	Description	Purpose
[62]	Synthetic dataset	This dataset has been devised to be used for machine learning training and evaluation purposes in several tasks of predictive modeling, classification, and clustering.	A goal for this dataset is the development, testing, and benchmarking of machine learning models
[42]	Synthetic dataset	This synthetic dataset is designed for machine learning training and evaluation.	The dataset serves multiple machine learning tasks, including Supervised Learning, Feature Engineering, and Model Evaluation.
[70]	Dataset Generated by Own Mathematical Model	This artificial data has been generated from a specific mathematical model to imitate real-world data distributions.	The generated database is mostly useful for machine-learning applications such as super-final learning, feature engineering, model evaluation, and mathematical model testing.
[74]	Historical Dataset to train a Random Forest Regression Prediction Model	This historical data set was designed for training any Random Forest Regression modeling for prediction.	This dataset can serve multiple purposes: for regression modeling, for feature importance analysis, for benchmarking of models with other datasets, and for historical data analysis.
[73]	Mathematical Model	The dataset is generated according to a mathematical model that simulates data patterns found in the real world to train machine-learning models based on regression.	The purpose of the dataset is for machine learning applications with an emphasis on applications such as regression modeling, feature engineering models, benchmarking predictive, and analyzing mathematical model validation.
[107]	GAIA Open dataset (Global Astrometry Interferometer for Astrophysics)	The GAIA Open Dataset is designed for machine-learning applications by depending on astronomical data that supports highly advanced predictive modeling.	This dataset is created for machine-learning applications regarding astrophysical predictions, regression modeling, feature generation, scientific inquiry, and model benchmarking.

TABLE 20. AI and machine learning-based tools.

Tool	Description	Use Cases	Advantage	Strength	Weaknesses
TensorFlow [124]/PyTorch [125]	Machine learning framework to predict offloading strategies.	Reinforcement learning-based resource allocation.	Open-source, highly scalable, GPU/TPU support	Excellent for deep learning tasks; large ecosystem and community.	Steep learning curve; resource-intensive for large models.
Scikit-Learn [126]	A machine learning library for classification and optimization in Python.	Analyses offloading success, and workload prediction.	Simple API, efficient for classical ML tasks	Lightweight and easy to integrate; fast prototyping.	Not suitable for deep learning or high-dimensional data
Deep Reinforcement Learning Algorithm	Includes DQN, PPO, and A3C for intelligent offloading decisions.	Dynamic task scheduling, energy-aware offloading.	Adaptable to dynamic environments; learns optimal policies.	Handles complex decision-making in real time.	Requires large training data and computation; can be unstable or slow to converge.

TABLE 21. Simulation and performance evaluation tools.

Tool	Description	Use Cases	Advantage	Strength	Weaknesses
CloudSim [127]	A cloud computing simulator modeling and simulation for offloading in cloud, edge, and fog environments.	Performance benchmarking, resource allocation analysis.	Flexible, open source, and customizable.	Widely adopted in cloud computing research; supports various configuration	Limited support for mobility and fog computing; lacks real-time capabilities.
iFogSim [128]	An extension of CloudSim in fog and edge computing simulation, which has taken IoT applications in developing various work scenarios.	Task scheduling, energy consumption analysis.	Tailored for IoT and fog/edge scenarios; includes energy models.	Effective in modeling latency-sensitive tasks.	Increased complexity; limited support for network heterogeneity.
EdgeCloudSim [129]	The scenarios focus on edge computing and consider mobility and network heterogeneity.	Evaluating MEC-based offloading strategies.	Incorporates mobility and dynamic network parameters.	Designed specifically for MEC scenarios; realistic modeling.	Less community support and documentation compared to CloudSim
Matlab/Simulink [130]	Computational modeling for task scheduling and offloading strategies.	Mathematical optimization, ML-based decision models.	Powerful for algorithm development and system-level simulation.	Rich library of tool-boxes; ideal for academic prototyping.	Licensing cost; not ideal for large-scale simulation or real-time systems.

TABLE 22. Energy and resource management tools.

Tool	Description	Use Cases	Advantage	Strength	Weaknesses
PowerTOP [131]	A Linux-based tool for analyzing energy consumption of tasks.	Energy profiling of offloading scenarios.	Easy to use; provides detailed power analysis.	Real-time energy monitoring; broad linux compatibility.	Limited to Linux; may not provide in-depth insights on all hardware.
Intel RAPL (Running Average Power Limit) [132]	Measures CPU power consumption in offloading tasks.	Energy-efficient task scheduling.	Low overhead; accurate CPU power usage measurement.	Hardware-level power measurement; good integration with Intel CPUs.	Intel-only support; not all platforms may expose all features.
Energy-aware Task Offloading Algorithms	Algorithms that optimize computation offloading based on energy constraints.	Mobile battery life extension, green computing.	Can significantly improve energy efficiency and performance.	Adaptive to different energy profiles; scalable for mobile/cloud.	Complexity in algorithm design and deployment.

TABLE 23. Security and privacy analysis tools.

Tool	Description	Use Cases	Advantage	Strength	Weaknesses
Wireshark [133]	A network analysis tool for monitoring offloaded data packets.	Detecting network vulnerabilities.	Open-source; detailed packet inspection.	Real-time traffic analysis; broad protocol support.	Requires expertise; not ideal for encrypted traffic.
Crypto Libraries (OpenSSL, PyCryptodome) [134]	Provides encryption mechanisms for secure offloading.	Ensuring data integrity in cloud-edge environments.	Widely used; supports many cryptographic standards.	Strong encryption support, flexible integration with applications.	Complexity in correct implementation; potential for misuse or misconfiguration.
Blockchain-Based Offloading [88]	Uses distributed ledger technology for trust-based offloading.	Secure offloading in multi-user environments.	Enhances trust and transparency.	Tamper-resistant data structure; decentralized security.	High computational overhead; latency issues, scalability concerns.

TABLE 24. 5 most used hardware tools in computation offloading

Tool	Description	Use Cases	Advantage	Strength	Weaknesses
NVIDIA Jetson series (Nano, Xavier) [142]	Embedded AI platform with CPU+GPU designed for edge small-scale training.	Real-time object detection, autonomous drones/robots, smart cameras.	Low latency, high AI throughput for its size.	Strong CUDA/cuDNN support, large developer community	Higher power draw than microcontroller-based system; limited battery life in mobile settings.
Google Coral Edge TPU [143]	ASIC accelerator optimized for TensorFlow Lite models on edge devices.	IoT vision, audio, and anomaly detection tasks with ultra-low latency.	Very low power consumption, compact size	Excellent inference speed for quantized models; good integration with Tensorflow Lite.	Limited to INT8 quantized models; not suitable for training.
Intel Movidius Myriad X [144]	VPU (Vision processing unit) optimized for computer vision and AI workloads.	Smart surveillance industrial vision, wearable AI camera	Efficient for vision pipeline, low power usage	Dedicated vision accelerator; USB form factor for flexibility.	Limited deep learning framework compatibility; less suited for non-vision AI tasks.
ARM Cortex-M + NPU (Ethos-U55) [145]	MCU with integrated neural processing unit for TinyML.	Sensor data processing, keyword spotting, low-power IoT AI.	Ultra-low power designed for always-on AI tasks.	Energy-efficient ML inference in sub-1 mW range.	Restricted memory and computer power; only support small models.
NVIDIA A100 GPU [146]	Data center GPU with Tensor cores for large-scale AI training and inference.	Cloud-based model training, high-performance computing offloading.	Industry-leading AI training performance.	Exceptional parallelism, supports mixed precision for speed	High cost and power consumption; unsuitable for edge/mobile deployment.

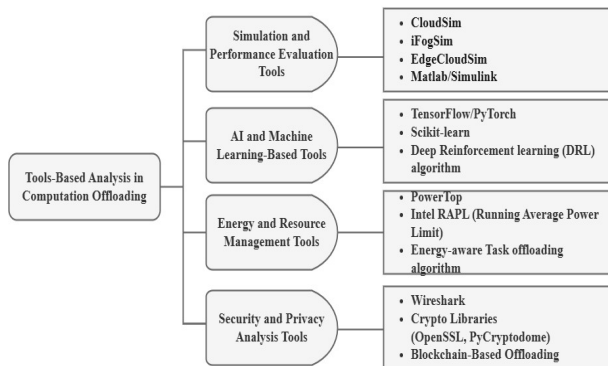


FIGURE 23. A categorized overview of tools for computation offloading.

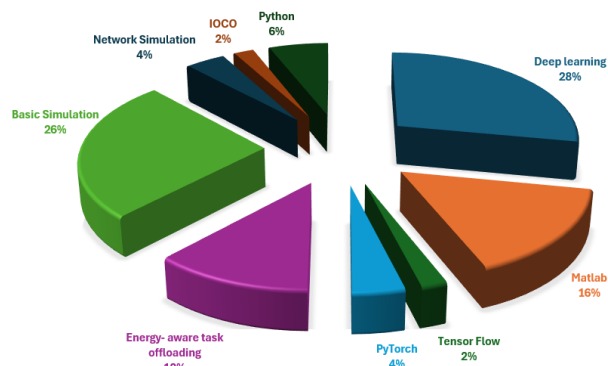


FIGURE 24. Leading simulation tools for computation offloading.

on computation offloading and categorized them into three categories: supervised learning, unsupervised learning, mathematical model approaches, reinforcement learning, and deep learning. These analyses were performed based on all performance metrics. We analyzed case studies, applied utilization techniques, developed formulations, and employed tools. It talks about how machine learning models are used in computation offloading to improve task offloading efficiency, lower latency, and resource allocation. The survey shows that offloading strategies can enhance the system's performance by dynamically adjusting to network conditions, user demand, and computational needs. Finally, it became evident that the majority of the studies in the literature focused on the RL approach. The second position is based on deep learning techniques. The least appealing is the mathematical model approach employed in the research. Energy consumption and cost are frequently mentioned in the research papers. Among the simulations, as shown in Figure 24, deep learning with 28% and Basic simulation with 26% seem to take priority: hence, the pivot plays its prime role. Matlab(16%) is largely a modeling tool; Energy-aware Task offloading(12%) fits well within the optimization of efficiency. Python(6%), PyTorch(4%), and TensorFlow(2%) are on the list of applications focusing on ML. Network simulation(4%) mainly assists with performance testing. Deep learning, simulation, and energy efficiency have taken up much time. The review underscores the importance of hybrid approaches integrating different ML-based optimization techniques to enhance decision-making in computation offloading. Some of the following future research directions and limitations in computation offloading:

- Advancing reinforcement learning algorithm: Ongoing refinement of the RL-based model is crucial to tackle challenges related to scalability, stability, and convergence, particularly in real-world mobile edge computing deployment.
- Integration with emerging technologies: By integrating 5G and 6G networks, federated learning, and blockchain, it is possible to create secure, decentralized, and ultra-low-latency offloading environments.
- Privacy-preserving offloading: In order to ensure the protection of sensitive data, future endeavors should focus on implementing robust mechanisms, such as differential privacy or secure multiparty computation, to safeguard data during the transmission and execution of tasks.
- Interoperable offloading framework: Creating platform-independent offloading solutions that can work effectively across diverse cloud, edge, and fog infrastructures is essential for wider deployment and improving efficiency.
- Energy-aware and context-aware optimization: More research is required on energy-efficient, context-aware offloading mechanisms that adaptively adjust based on battery level, network conditions, and user behavior.

REFERENCES

- [1] L. Lin, X. Liao, H. Jin and P. Li, "Computation Offloading Toward Edge Computing," in *Proceedings of the IEEE*, vol. 107, no. 8, pp. 1584-1607, Aug. 2019, doi: 10.1109/JPROC.2019.2922285.
- [2] S. Zhang, N. Yi and Y. Ma, "A Survey of Computation Offloading With Task Types," in *IEEE Transactions on Intelligent Transportation Systems*, vol. 25, no. 8, pp. 8313-8333, Aug. 2024, doi: 10.1109/TITS.2024.3410896.
- [3] P. Mach and Z. Becvar, "Mobile Edge Computing: A Survey on Architecture and Computation Offloading," in *IEEE Communications Surveys & Tutorials*, vol. 19, no. 3, pp. 1628-1656, thirdquarter 2017, doi: 10.1109/COMST.2017.2682318.
- [4] K. Zhang et al., "Energy-Efficient Offloading for Mobile Edge Computing in 5G Heterogeneous Networks," in *IEEE Access*, vol. 4, pp. 5896-5907, 2016, doi: 10.1109/ACCESS.2016.2597169.
- [5] S. Wang, X. Zhang, Y. Zhang, L. Wang, J. Yang and W. Wang, "A Survey on Mobile Edge Networks: Convergence of Computing, Caching and Communications," in *IEEE Access*, vol. 5, pp. 6757-6779, 2017, doi: 10.1109/ACCESS.2017.2685434.
- [6] X. Chen, L. Jiao, W. Li and X. Fu, "Efficient Multi-User Computation Offloading for Mobile-Edge Cloud Computing," in *IEEE/ACM Transactions on Networking*, vol. 24, no. 5, pp. 2795-2808, October 2016, doi: 10.1109/TNET.2015.2487344.
- [7] X. Wang and H. Guo, "Mobility-Aware Computation Offloading for Swarm Robotics using Deep Reinforcement Learning," 2021 IEEE 18th Annual Consumer Communications & Networking Conference (CCNC), Las Vegas, NV, USA, 2021, pp. 1-4, doi: 10.1109/CCNC49032.2021.9369456.
- [8] Y. Guo, Z. Mi, Y. Yang and M. S. Obaidat, "An Energy Sensitive Computation Offloading Strategy in Cloud Robotic Network Based on GA," in *IEEE Systems Journal*, vol. 13, no. 3, pp. 3513-3523, Sept. 2019, doi: 10.1109/JSYST.2018.2830395.
- [9] M. Chen, W. Saad, C. Yin and M. Debbah, "Data Correlation-Aware Resource Management in Wireless Virtual Reality (VR): An Echo State Transfer Learning Approach," in *IEEE Transactions on Communications*, vol. 67, no. 6, pp. 4267-4280, June 2019, doi: 10.1109/TCOMM.2019.2900624.
- [10] J. Du, F. R. Yu, G. Lu, J. Wang, J. Jiang and X. Chu, "MEC-Assisted Immersive VR Video Streaming Over Terahertz Wireless Networks: A Deep Reinforcement Learning Approach," in *IEEE Internet of Things Journal*, vol. 7, no. 10, pp. 9517-9529, Oct. 2020, doi: 10.1109/JIOT.2020.3003449.
- [11] B. Liu, C. Liu and M. Peng, "Computation Offloading and Resource Allocation in Unmanned Aerial Vehicle Networks," in *IEEE Transactions on Vehicular Technology*, vol. 72, no. 4, pp. 4981-4995, April 2023, doi: 10.1109/TVT.2022.3222907.
- [12] Z. Yu, Y. Gong, S. Gong and Y. Guo, "Joint Task Offloading and Resource Allocation in UAV-Enabled Mobile Edge Computing," in *IEEE Internet of Things Journal*, vol. 7, no. 4, pp. 3147-3159, April 2020, doi: 10.1109/JIOT.2020.2965898.
- [13] M. Dai, Z. Su, Q. Xu and N. Zhang, "Vehicle Assisted Computing Offloading for Unmanned Aerial Vehicles in Smart City," in *IEEE Transactions on Intelligent Transportation Systems*, vol. 22, no. 3, pp. 1932-1944, March 2021, doi: 10.1109/TITS.2021.3052979.
- [14] K. Xiong, S. Leng, C. Huang, C. Yuen and Y. L. Guan, "Intelligent Task Offloading for Heterogeneous V2X Communications," in *IEEE Transactions on Intelligent Transportation Systems*, vol. 22, no. 4, pp. 2226-2238, April 2021, doi: 10.1109/TITS.2020.3015210.
- [15] Y. Liu et al., "Physical Layer Security Assisted Computation Offloading in Intelligently Connected Vehicle Networks," in *IEEE Transactions on Wireless Communications*, vol. 20, no. 6, pp. 3555-3570, June 2021, doi: 10.1109/TWC.2021.3051772.
- [16] U. A. Khan, W. Khalid and S. Saifullah, "Energy Efficient Resource Allocation and Computation Offloading Strategy in a UAV-enabled Secure Edge-Cloud Computing System," 2020 IEEE International Conference on Smart Internet of Things (SmartIoT), Beijing, China, 2020, pp. 58-63, doi: 10.1109/SmartIoT49966.2020.00018.
- [17] F. Sufyan, M. S. Chishti and A. Banerjee, "Energy and Delay Aware Computation Offloading Scheme in MCC Environment," 2022 5th Conference on Cloud and Internet of Things (CIoT), Marrakech, Morocco, 2022, pp. 247-254, doi: 10.1109/CIoT53061.2022.9766509.
- [18] F. Sufyan and A. Banerjee, "Computation Offloading for Distributed Mobile Edge Computing Network: A Multiobjective Approach," in *IEEE Access*, vol. 8, pp. 149915-149930, 2020, doi: 10.1109/ACCESS.2020.3016046.
- [19] J. Liu, Y. Mao, J. Zhang and K. B. Letaief, "Delay-optimal computation task scheduling for mobile-edge computing systems," 2016 IEEE International Symposium on Information Theory (ISIT), Barcelona, Spain, 2016, pp. 1451-1455, doi: 10.1109/ISIT.2016.7541539.
- [20] B. Li, Z. Fei, J. Shen, X. Jiang and X. Zhong, "Dynamic Offloading for Energy Harvesting Mobile Edge Computing: Architecture, Case Studies, and Future Directions," in *IEEE Access*, vol. 7, pp. 79877-79886, 2019, doi: 10.1109/ACCESS.2019.2922362.
- [21] N. Abbas, Y. Zhang, A. Taherkordi and T. Skeie, "Mobile Edge Computing: A Survey," in *IEEE Internet of Things Journal*, vol. 5, no. 1, pp. 450-465, Feb. 2018, doi: 10.1109/JIOT.2017.2750180.
- [22] T. Taleb, K. Samdanis, B. Mada, H. Flinck, S. Dutta and D. Sabella, "On Multi-Access Edge Computing: A Survey of the Emerging 5G Network Edge Cloud Architecture and Orchestration," in *IEEE Communications Surveys & Tutorials*, vol. 19, no. 3, pp. 1657-1681, thirdquarter 2017, doi: 10.1109/COMST.2017.2705720.
- [23] Y. Mao, C. You, J. Zhang, K. Huang and K. B. Letaief, "A Survey on Mobile Edge Computing: The Communication Perspective," in *IEEE Communications Surveys & Tutorials*, vol. 19, no. 4, pp. 2322-2358, Fourthquarter 2017, doi: 10.1109/COMST.2017.2745201.
- [24] P. Mach and Z. Becvar, "Mobile Edge Computing: A Survey on Architecture and Computation Offloading," in *IEEE Communications Surveys & Tutorials*, vol. 19, no. 3, pp. 1628-1656, thirdquarter 2017, doi: 10.1109/COMST.2017.2682318.
- [25] Akhirl Islam, Arindam Debnath, Manojit Ghose, Suchetana Chakraborty, A Survey on Task Offloading in Multi-access Edge Computing, *Journal of Systems Architecture*, Volume 118, 2021, 102225, ISSN 1383-7621, <https://doi.org/10.1016/j.sysarc.2021.102225>.
- [26] Honghai Wu, Yixuan Lu, Huahong Ma, Ling Xing, Kaikai Deng, Xiaoying Lu, A survey on task type-based computation offloading in mobile edge networks, *Ad Hoc Networks*, Volume 169, 2025, 103754, ISSN 1570-8705, <https://doi.org/10.1016/j.adhoc.2025.103754>.
- [27] Hortelano, D., de Miguel, I., Barroso, R. J. D., Aguado, J. C., Merayo, N., Ruiz, L., ... & Abril, E. J. (2023). A comprehensive survey on reinforcement-learning-based computation offloading techniques in edge computing systems. *Journal of Network and Computer Applications*, 216, 103669, doi:10.1016/j.jnca.2023.103669.
- [28] C. Luo, J. Zhang, X. Cheng, Y. Hong, Z. Chen and X. Xing, "Computation Off-Loading in Resource-Constrained Edge Computing Systems Based on

- Deep Reinforcement Learning," in *IEEE Transactions on Computers*, vol. 73, no. 1, pp. 109-122, Jan. 2024, doi: 10.1109/TC.2023.3321938.
- [29] Chen, Z., & Wang, X. (2020). Decentralized computation offloading for multi-user mobile edge computing: a deep reinforcement learning approach. *EURASIP Journal on Wireless Communications and Networking*, 2020(1). doi: 10.1186/s13638-020-01801-6.
- [30] E. Cuervo, A. Balasubramanian, D. K. Cho, A. Wolman, S. Saroiu, R. Chandra, and P. Bahl, "MAUI: Making smartphones last longer with code offload," in **Proceedings of the 8th International Conference on Mobile Systems, Applications, and Services (MobiSys)**, San Francisco, CA, USA, Jun. 2010, pp. 49-62. doi: 10.1145/1814433.1814441.
- [31] B. G. Chun, S. Ihm, P. Maniatis, M. Naik, and A. Patti, "CloneCloud: Elastic execution between mobile device and cloud," in **Proceedings of the 6th Conference on Computer Systems (EuroSys)**, Salzburg, Austria, Apr. 2011, pp. 301-314. doi: 10.1145/1966445.1966473.
- [32] Sun, S., Zhu, Q. Joint optimization algorithm of offloading decision and resource allocation based on integrated sensing, communication, and computation. *Wireless Netw* 30, 557-576 (2024). <https://doi.org/10.1007/s11276-023-03498-z>
- [33] J. Bi, K. Zhang, H. Yuan and J. Zhang, "Energy-Efficient Computation Offloading for Static and Dynamic Applications in Hybrid Mobile Edge Cloud System," in *IEEE Transactions on Sustainable Computing*, vol. 8, no. 2, pp. 232-244, 1 April-June 2023, doi: 10.1109/TSUSC.2022.3216461.
- [34] S. Guo, J. Liu, Y. Yang, B. Xiao and Z. Li, "Energy-Efficient Dynamic Computation Offloading and Cooperative Task Scheduling in Mobile Cloud Computing," in *IEEE Transactions on Mobile Computing*, vol. 18, no. 2, pp. 319-333, 1 Feb. 2019, doi: 10.1109/TMC.2018.2831230.
- [35] Choudhury, A., Ghose, M., & Islam, A. (2024). Machine learning-based computation offloading in multi-access edge computing: A survey. *Journal of Systems Architecture*, 103090, doi: 10.1016/j.sysarc.2024.103090.
- [36] P. R. Lutui, B. Cusack and G. Maeakafa, "Energy efficiency for IoT devices in home environments," 2018 IEEE International Conference on Environmental Engineering (EE), Milan, Italy, 2018, pp. 1-6, doi: 10.1109/EEI.2018.8385277.
- [37] R. P. Mathur and M. Sharma, "A survey on computational offloading in mobile cloud computing," 2019 Fifth International Conference on Image Information Processing (ICIIP), Shimla, India, 2019, pp. 515-520, doi: 10.1109/ICIIP47207.2019.8985893.
- [38] Q. Zhang, L. Gui, F. Hou, J. Chen, S. Zhu and F. Tian, "Dynamic Task Offloading and Resource Allocation for Mobile-Edge Computing in Dense Cloud RAN," in *IEEE Internet of Things Journal*, vol. 7, no. 4, pp. 3282-3299, April 2020, doi: 10.1109/JIOT.2020.2967502.
- [39] X. Chen, J. Cao, Y. Sahni, S. Jiang and Z. Liang, "Dynamic Task Offloading in Edge Computing Based on Dependency-Aware Reinforcement Learning," in *IEEE Transactions on Cloud Computing*, vol. 12, no. 2, pp. 594-608, April-June 2024, doi: 10.1109/TCC.2024.3381646.
- [40] Lingayya, S., Jodumutt, S.B., Pawar, S.R. et al. Dynamic task offloading for resource allocation and privacy-preserving framework in Kubeedge-based edge computing using machine learning. *Cluster Comput* 27, 9415-9431 (2024). <https://doi.org/10.1007/s10586-024-04420-8>
- [41] Shi Dong, Junxiao Tang, Khushnood Abbas, Ruizhe Hou, Joarder Kamruzzaman, Leszek Rutkowski, Rajkumar Buyya, Task offloading strategies for mobile edge computing: A survey, *Computer Networks*, Volume 254, 2024, 110791, ISSN 1389-1286, <https://doi.org/10.1016/j.comnet.2024.110791>.
- [42] S. Yu, X. Wang and R. Langar, "Computation offloading for mobile edge computing: A deep learning approach," 2017 IEEE 28th Annual International Symposium on Personal, Indoor, and Mobile Radio Communications (PIMRC), Montreal, QC, Canada, 2017, pp. 1-6, doi: 10.1109/PIMRC.2017.8292514.
- [43] L. Jiao, R. Friedman, X. Fu, S. Secci, Z. Smoreda, and H. Tschofenig, "Cloud-based computation offloading for mobile devices: State of the art, challenges and opportunities," 2013 Future Network & Mobile Summit, Lisboa, Portugal, 2013, pp. 1-11.
- [44] S. Yu, R. Langar, X. Fu, L. Wang and Z. Han, "Computation Offloading With Data Caching Enhancement for Mobile Edge Computing," in *IEEE Transactions on Vehicular Technology*, vol. 67, no. 11, pp. 11098-11112, Nov. 2018, doi: 10.1109/TVT.2018.2869144.
- [45] Taheri-abed, S., Eftekhari Moghadam, A.M. & Rezvani, M.H. Machine learning-based computation offloading in edge and fog: a systematic review. *Cluster Comput* 26, 3113-3144 (2023). <https://doi.org/10.1007/s10586-023-04100-z>.
- [46] S. Bi, L. Huang and Y. -J. A. Zhang, "Joint Optimization of Service Caching Placement and Computation Offloading in Mobile Edge Computing Systems," in *IEEE Transactions on Wireless Communications*, vol. 19, no. 7, pp. 4947-4963, July 2020, doi: 10.1109/TWC.2020.2988386.
- [47] W. Gao, Z. Yu, L. Wang, H. Cui, B. Guo and H. Xiong, "Hierarchical Deep Reinforcement Learning for Computation Offloading in Autonomous Multi-Robot Systems," in *IEEE Robotics and Automation Letters*, vol. 10, no. 1, pp. 540-547, Jan. 2025, doi: 10.1109/LRA.2024.3511408.
- [48] S. Wang, Y. Ruan, Y. Tu, S. Wagle, C. G. Brinton and C. Joe-Wong, "Network-Aware Optimization of Distributed Learning for Fog Computing," in *IEEE/ACM Transactions on Networking*, vol. 29, no. 5, pp. 2019-2032, Oct. 2021, doi: 10.1109/TNET.2021.3075432.
- [49] J. Hu, Y. Li, G. Zhao, B. Xu, Y. Ni and H. Zhao, "Deep Reinforcement Learning for Task Offloading in Edge Computing Assisted Power IoT," in *IEEE Access*, vol. 9, pp. 93892-93901, 2021, doi: 10.1109/ACCESS.2021.3092381.
- [50] Arash Heidari, Nima Jafari Navimipour, Mohammad Ali Jabraeil Jamali, Shahin Akbarpour, A green, secure, and deep intelligent method for dynamic IoT-edge-cloud offloading scenarios, *Sustainable Computing: Informatics and Systems*, Volume 38,2023,100859, ISSN 2210-5379, <https://doi.org/10.1016/j.suscom.2023.100859>.
- [51] Congfeng Jiang, Tiantian Fan, Honghao Gao, Weisong Shi, Liangkai Liu, Christophe Cérin, Jian Wan, Energy aware edge computing: A survey, *Computer Communications*, Volume 151, 2020, Pages 556-580, ISSN 0140-3664, <https://doi.org/10.1016/j.comcom.2020.01.004>.
- [52] Zhenqi Huang, Zhufang Kuang, Bin Xu et al. Dependency-aware Task Collaborative Offloading and Resource Allocation in UAV Enabled Edge Computing, 15 November 2024, PREPRINT (Version 1) available at Research Square [<https://doi.org/10.21203/rs.3.rs-4689562/v1>]
- [53] M. Gao, W. Cui, D. Gao, R. Shen, J. Li and Y. Zhou, "Deep Neural Network Task Partitioning and Offloading for Mobile Edge Computing," 2019 IEEE Global Communications Conference (GLOBECOM), Waikoloa, HI, USA, 2019, pp. 1-6, doi: 10.1109/GLOBECOM38437.2019.9013404.
- [54] J. H. Anajemba, T. Yue, C. Iwendi, M. Alenezi and M. Mittal, "Optimal Cooperative Offloading Scheme for Energy Efficient Multi-Access Edge Computation," in *IEEE Access*, vol. 8, pp. 53931-53941, 2020, doi: 10.1109/ACCESS.2020.2980196.
- [55] S. Chouikhi, M. Esseghir and L. Merghem-Boulahia, "Energy-Efficient Computation Offloading Based on Multiagent Deep Reinforcement Learning for Industrial Internet of Things Systems," in *IEEE Internet of Things Journal*, vol. 11, no. 7, pp. 12228-12239, 1 April, 2024, doi: 10.1109/JIOT.2023.3333044.
- [56] T. X. Tran and D. Pompili, "Joint Task Offloading and Resource Allocation for Multi-Server Mobile-Edge Computing Networks," in *IEEE Transactions on Vehicular Technology*, vol. 68, no. 1, pp. 856-868, Jan. 2019, doi: 10.1109/TVT.2018.2881191.
- [57] A. Madej, N. Wang, N. Athanasopoulos, R. Ranjan and B. Varghese, "Priority-based Fair Scheduling in Edge Computing," 2020 IEEE 4th International Conference on Fog and Edge Computing (ICFEC), Melbourne, VIC, Australia, 2020, pp. 39-48, doi: 10.1109/ICFEC50348.2020.00012.
- [58] T. Alfakih, M. M. Hassan, A. Gumaei, C. Savaglio and G. Fortino, "Task Offloading and Resource Allocation for Mobile Edge Computing by Deep Reinforcement Learning Based on SARSA," in *IEEE Access*, vol. 8, pp. 54074-54084, 2020, doi: 10.1109/ACCESS.2020.2981434.
- [59] S. Nath and J. Wu, "Deep reinforcement learning for dynamic computation offloading and resource allocation in cache-assisted mobile edge computing systems," in *Intelligent and Converged Networks*, vol. 1, no. 2, pp. 181-198, Sept. 2020, doi: 10.23919/ICN.2020.0014.
- [60] S. Birhanu Engidayehu, T. Mahboob and M. Young Chung, "Deep Reinforcement Learning-based Task Offloading and Resource Allocation in MEC-enabled Wireless Networks," 2022 27th Asia Pacific Conference on Communications (APCC), Jeju Island, Korea, Republic of, 2022, pp. 226-230, doi: 10.1109/APCC55198.2022.9943689.
- [61] H. Wu, "Multi-Objective Decision-Making for Mobile Cloud Offloading: A Survey," in *IEEE Access*, vol. 6, pp. 3962-3976, 2018, doi: 10.1109/ACCESS.2018.2791504.
- [62] A. Crutcher, C. Koch, K. Coleman, J. Patman, F. Esposito and P. Calyam, "Hyperprofile-Based Computation Offloading for Mobile Edge Networks," 2017 IEEE 14th International Conference on Mobile Ad Hoc and Sensor Systems (MASS), Orlando, FL, USA, 2017, pp. 525-529, doi: 10.1109/MASS.2017.91.
- [63] H. Wu, W. J. Knottenbelt and K. Wolter, "An Efficient Application Partitioning Algorithm in Mobile Environments," in *IEEE Transactions*

- on Parallel and Distributed Systems, vol. 30, no. 7, pp. 1464-1480, 1 July 2019, doi: 10.1109/TPDS.2019.2891695.
- [64] Y. Cui, Y. Liang and R. Wang, "Intelligent Task Offloading Algorithm for Mobile Edge Computing in Vehicular Networks," 2020 IEEE 91st Vehicular Technology Conference (VTC2020-Spring), Antwerp, Belgium, 2020, pp. 1-5, doi: 10.1109/VTC2020-Spring48590.2020.9128803.
- [65] A. A. Majeed, A. U. R. Khan, R. UlAmin, J. Muhammad and S. Ayub, "Code offloading using support vector machine," 2016 Sixth International Conference on Innovative Computing Technology (INTECH), Dublin, Ireland, 2016, pp. 98-103, doi: 10.1109/INTECH.2016.7845057.
- [66] X. Diao, J. Zheng, Y. Cai, X. Dong and X. Zhang, "Joint User Clustering, Resource Allocation and Power Control for NOMA-based Mobile Edge Computing," 2018 10th International Conference on Wireless Communications and Signal Processing (WCSP), Hangzhou, China, 2018, pp. 1-6, doi: 10.1109/WCSP.2018.8555861.
- [67] O. Ivanchenko, V. Kharchenko, B. Moroz, L. Kabak and K. Smoktii, "Semi-Markov availability model considering deliberate malicious impacts on an Infrastructure-as-a-Service Cloud," 2018 14th International Conference on Advanced Trends in Radioelectronics, Telecommunications and Computer Engineering (TCSET), Lviv-Slavske, Ukraine, 2018, pp. 570-573, doi: 10.1109/TCSET.2018.8336266.
- [68] A. Samir and C. Pahl, "DLA: Detecting and Localizing Anomalies in Containerized Microservice Architectures Using Markov Models," 2019 7th International Conference on Future Internet of Things and Cloud (FiCloud), Istanbul, Turkey, 2019, pp. 205-213, doi: 10.1109/FiCloud.2019.00036.
- [69] Sheng, J., Hu, J., Teng, X., Wang, B., & Pan, X. (2019). Computation Offloading Strategy in Mobile Edge Computing. Information, 10(6), 191. <https://doi.org/10.3390/info10060191>
- [70] Z. Ali, L. Jiao, T. Baker, G. Abbas, Z. H. Abbas and S. Khaf, "A Deep Learning Approach for Energy Efficient Computational Offloading in Mobile Edge Computing," in IEEE Access, vol. 7, pp. 149623-149633, 2019, doi: 10.1109/ACCESS.2019.2947053.
- [71] J. Bi, H. Yuan, S. Duanmu, M. Zhou and A. Abusorrah, "Energy-Optimized Partial Computation Offloading in Mobile-Edge Computing With Genetic Simulated-Annealing-Based Particle Swarm Optimization," in IEEE Internet of Things Journal, vol. 8, no. 5, pp. 3774-3785, 1 March 1, 2021, doi: 10.1109/IIOT.2020.3024223.
- [72] W. -J. Feng, C. -H. Yang and X. -S. Zhou, "Multi-User and Multi-Task Offloading Decision Algorithms Based on Imbalanced Edge Cloud," in IEEE Access, vol. 7, pp. 95970-95977, 2019, doi: 10.1109/ACCESS.2019.2928377.
- [73] G. Yoon, G. -Y. Kim, H. Yoo, S. C. Kim and R. Kim, "Implementing Practical DNN-Based Object Detection Offloading Decision for Maximizing Detection Performance of Mobile Edge Devices," in IEEE Access, vol. 9, pp. 140199-140211, 2021, doi: 10.1109/ACCESS.2021.3118731.
- [74] X. Chen, M. Li, H. Zhong, Y. Ma and C. -H. Hsu, "DNNOff: Offloading DNN-Based Intelligent IoT Applications in Mobile Edge Computing," in IEEE Transactions on Industrial Informatics, vol. 18, no. 4, pp. 2820-2829, April 2022, doi: 10.1109/TII.2021.3075464.
- [75] Gong, Y., Lv, C., Cao, S., Yan, L., & Wang, H. (2020). Deep learning-based computation offloading with energy and performance optimization. EURASIP Journal on Wireless Communications and Networking, 2020, 1-8, doi: 10.1186/s13638-020-01678-5.
- [76] M. Min, L. Xiao, Y. Chen, P. Cheng, D. Wu and W. Zhuang, "Learning-Based Computation Offloading for IoT Devices With Energy Harvesting," in IEEE Transactions on Vehicular Technology, vol. 68, no. 2, pp. 1930-1941, Feb. 2019, doi: 10.1109/TVT.2018.2890685.
- [77] @articleZHANG2019111, title = Task migration for mobile edge computing using deep reinforcement learning, journal = Future Generation Computer Systems, volume = 96, pages = 111-118, year = 2019, issn = 0167-739X, doi = <https://doi.org/10.1016/j.future.2019.01.059>, author = Cheng Zhang and Zixuan Zheng
- [78] Meng, H., Chao, D., & Guo, Q. (2019, April). Deep reinforcement learning based task offloading algorithm for mobile-edge computing systems. In Proceedings of the 2019 4th International Conference on Mathematics and Artificial Intelligence (pp. 90-94), doi = 10.1145/3325730.3325732
- [79] L. Huang, S. Bi and Y. -J. A. Zhang, "Deep Reinforcement Learning for Online Computation Offloading in Wireless Powered Mobile-Edge Computing Networks," in IEEE Transactions on Mobile Computing, vol. 19, no. 11, pp. 2581-2593, 1 Nov. 2020, doi: 10.1109/TMC.2019.2928811.
- [80] H. Zhang, Y. Yang, X. Huang, C. Fang and P. Zhang, "Ultra-Low Latency Multi-Task Offloading in Mobile Edge Computing," in IEEE Access, vol. 9, pp. 32569-32581, 2021, doi: 10.1109/ACCESS.2021.3061105.
- [81] C. Li, C. Deng, Y. Zhang, and S. Wan, "Federated Meta-Learning Based Computation Offloading Approach with Energy-Delay Tradeoffs in UAV-assisted VEC," in IEEE Transactions on Mobile Computing, doi: 10.1109/TMC.2025.3573278.
- [82] A. D. Raha, A. Adhikary, M. Gain and C. S. Hong, "A Federated Learning Framework for Optimizing Edge Computing with Semantic Offloading," 2025 27th International Conference on Advanced Communications Technology (ICACT), Pyeong Chang, Korea, Republic of, 2025, pp. 142-149, doi: 10.23919/ICACT63878.2025.10936753.
- [83] J. Zhang, F. Huang, S. Zhu, and X. Xiao, "A Resource Allocation Strategy in Internet of Vehicles Based on Multi-Task Federated Learning and Incentive Mechanism," in IEEE Transactions on Intelligent Transportation Systems, doi: 10.1109/TITS.2025.3528969.
- [84] X. Chen, B. Xiao, X. Lin, Z. Chen and G. Min, "Multi-agent Collaboration for Vehicular Task Offloading Using Federated Deep Reinforcement Learning," in IEEE Transactions on Mobile Computing, doi: 10.1109/TMC.2025.3557898.
- [85] Rahmati, M. Energy-aware federated learning for secure edge computing in 5G-enabled IoT networks. Journal of Electrical Systems and Inf Technol 12, 13 (2025). <https://doi-org.egateway.chennai.vit.ac.in/10.1186/s43067-025-00203-2>.
- [86] N. Agarwal and S. Joshi, "Federated Learning-Based Task Offloading in a UAV-Aided Cloud Computing Mobile Network," in IEEE Transactions on Vehicular Technology, vol. 73, no. 10, pp. 15751-15756, Oct. 2024, doi: 10.1109/TVT.2024.3404223.
- [87] Matheen Fathima G, Shakkeera L, Efficient task scheduling and computational offloading optimization with federated learning and blockchain in mobile cloud computing, Results in Control and Optimization, Volume 18, 2025, 100524, ISSN 2666-7207, <https://doi.org/10.1016/j.rico.2025.100524>. (<https://www.sciencedirect.com/science/article/pii/S2666720725000104>).
- [88] Reis, Manuel J. C. S. 2025. "Blockchain-Enhanced Security for 5G Edge Computing in IoT" Computation 13, no. 4: 98. <https://doi.org/10.3390/computation13040098>
- [89] Zhaolong Gao, Wei Yan, The real-time data processing framework for blockchain and edge computing, Alexandria Engineering Journal, Volume 120, 2025, Pages 50-61, ISSN 1110-0168, <https://doi.org/10.1016/j.aej.2025.01.092>. (<https://www.sciencedirect.com/science/article/pii/S111001682500119X>)
- [90] F. Xu, Z. Zhao, L. Liu, X. Yuan and Q. Pei, "Scalable Blockchain-empowered Distributed Computation Offloading: A Deep Reinforcement Learning Approach," IEEE INFOCOM 2024 - IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS), Vancouver, BC, Canada, 2024, pp. 1-6, doi: 10.1109/INFOCOMWKSHPS61880.2024.10620784.
- [91] Ting Chen, Shujiao Wang, Xin Fan, Xiujuan Zhang, Chuanwen Luo, Yi Hong, UAV-Assisted Multi-Object Computing Offloading for Blockchain-Enabled Vehicle-to-Everything Systems, Computers, Materials and Continua, Volume 81, Issue 3, 2024, Pages 3927-3950, ISSN 1546-2218, <https://doi.org/10.32604/cmc.2024.056961>. (<https://www.sciencedirect.com/science/article/pii/S1546221824008348>)
- [92] Yang, S., Lee, G., Huang, L. (2022). Deep Learning-Based Dynamic Computation Task offloading for mobile edge computing networks. Sensors, 22(11), 4088. <https://doi.org/10.3390/s22114088>.
- [93] Kar, B., Yahya, W., Lin, Y., Ali, A. (2022). A Survey on Offloading in Federated Cloud-Edge-Fog Systems with Traditional Optimization and Machine Learning. ArXiv. <https://arxiv.org/abs/2202.10628>.
- [94] Nguyen, D. C., Pathirana, P. N., Ding, M., Seneviratne, A. (2021). Secure computation offloading in blockchain based IoT networks with deep reinforcement learning. IEEE Transactions on Network Science and Engineering, 8(4), 3192-3208. <https://doi.org/10.1109/tNSE.2021.3106956>
- [95] Luo, Z., Dai, X. (2024). Reinforcement learning-based computation offloading in edge computing: Principles, methods, challenges. Alexandria Engineering Journal, 108, 89-107. <https://doi.org/10.1016/j.aej.2024.07.049>
- [96] Lim, W. Y. B., Luong, N. C., Hoang, D. T., Jiao, Y., Liang, Y., Yang, Q., Niyato, D., Miao, C. (2020). Federated Learning in Mobile Edge Networks: A Comprehensive survey. IEEE Communications Surveys Tutorials, 22(3), 2031-2063. <https://doi.org/10.1109/comst.2020.2986024>
- [97] Ma, M., Gong, C., Zeng, L., Yang, Y., Wu, L. (2024). FLOCOFF: Data Heterogeneity Resilient Federated Learning with Communication-Efficient Edge Offloading. IEEE Journal on Selected Areas in Communications, 42(11), 3262-3277. <https://doi.org/10.1109/jsac.2024.3431526>

- [98] Tang, M., Wong, V. W. (2020). Deep reinforcement learning for task offloading in mobile edge computing systems. *IEEE Transactions on Mobile Computing*, 21(6), 1985–1997. <https://doi.org/10.1109/tmc.2020.3036871>
- [99] L. Ale, N. Zhang, X. Fang, X. Chen, S. Wu and L. Li, "Delay-Aware and Energy-Efficient Computation Offloading in Mobile-Edge Computing Using Deep Reinforcement Learning," in *IEEE Transactions on Cognitive Communications and Networking*, vol. 7, no. 3, pp. 881–892, Sept. 2021, doi: 10.1109/TCCN.2021.3066619.
- [100] Moghaddasi, K., Rajabi, S. (2024). Blockchain-Enhanced Offloading in Mobile Edge Computing: A Systematic Review and Survey of Current Trends and Future Directions. *ArXiv*. <https://arxiv.org/abs/2403.05961>
- [101] Thangaraj, V., Sree, T. R. (2024). MSCO: Mobility-aware Secure Computation Offloading in blockchain-enabled Fog computing environments. *Journal of Cloud Computing Advances Systems and Applications*, 13(1). <https://doi.org/10.1186/s13677-024-00599-8>
- [102] T. Taleb, K. Samdanis, B. Mada, H. Flinck, S. Dutta and D. Sabella, "On Multi-Access Edge Computing: A Survey of the Emerging 5G Network Edge Cloud Architecture and Orchestration," in *IEEE Communications Surveys & Tutorials*, vol. 19, no. 3, pp. 1657–1681, thirdquarter 2017, doi: 10.1109/COMST.2017.2705720.
- [103] 3GPP. (2018). Management and orchestration; Concepts, use cases and requirements.
- [104] Kim, J., Kim, D., & Choi, S. (2017). 3GPP SA2 architecture and functions for 5G mobile communication system. *ICT Express*, 3(1), 1–8. <https://doi.org/10.1016/j.ict.2017.03.007>
- [105] Zhao, L., Zhou, G., Zheng, G., I. C., You, X., & Hanzo, L. (2021). Open-Source Multi-Access Edge Computing for 6G: Opportunities and challenges. *IEEE Access*, 9, 158426–158439. <https://doi.org/10.1109/access.2021.3130418>.
- [106] J. Xu, L. Chen and S. Ren, "Online Learning for Offloading and Autoscaling in Energy Harvesting Mobile Edge Computing," in *IEEE Transactions on Cognitive Communications and Networking*, vol. 3, no. 3, pp. 361–373, Sept. 2017, doi: 10.1109/TCCN.2017.2725277.
- [107] Q. Luo, C. Li, T. H. Luan and W. Shi, "Minimizing the Delay and Cost of Computation Offloading for Vehicular Edge Computing," in *IEEE Transactions on Services Computing*, vol. 15, no. 5, pp. 2897–2909, 1 Sept.–Oct. 2022, doi: 10.1109/TSC.2021.3064579.
- [108] Y. Wang, Z. Qian, L. He, R. Yin and C. Wu, "Intelligent Online Computation Offloading for Wireless-Powered Mobile-Edge Computing," in *IEEE Internet of Things Journal*, vol. 11, no. 17, pp. 28960–28974, 1 Sept. 1, 2024, doi: 10.1109/JIOT.2024.3406204.
- [109] M. Wu, Q. Song, L. Guo and L. Lee, "Energy-Efficient Secure Computation Offloading in Wireless Powered Mobile Edge Computing Systems," in *IEEE Transactions on Vehicular Technology*, vol. 72, no. 5, pp. 6907–6912, May 2023, doi: 10.1109/TVT.2023.3236327.
- [110] B. Qiu, Y. Wang, H. Xiao and Z. Zhang, "Deep Reinforcement Learning-Based Adaptive Computation Offloading and Power Allocation in Vehicular Edge Computing Networks," in *IEEE Transactions on Intelligent Transportation Systems*, vol. 25, no. 10, pp. 13339–13349, Oct. 2024, doi: 10.1109/ITITS.2024.3391831.
- [111] K. Li et al., "Computation Offloading in Resource-Constrained Multi-Access Edge Computing," in *IEEE Transactions on Mobile Computing*, vol. 23, no. 11, pp. 10665–10677, Nov. 2024, doi: 10.1109/TMC.2024.3383041.
- [112] Y. Chen, N. Zhang, Y. Zhang and X. Chen, "Dynamic Computation Offloading in Edge Computing for Internet of Things," in *IEEE Internet of Things Journal*, vol. 6, no. 3, pp. 4242–4251, June 2019, doi: 10.1109/JIOT.2018.2875715.
- [113] Z. Yang and W. Bai, "Distributed Computation Offloading in Mobile Fog Computing: A Deep Neural Network Approach," in *IEEE Communications Letters*, vol. 26, no. 3, pp. 696–700, March 2022, doi: 10.1109/LCOMM.2021.3138800.
- [114] J. Zhou, Q. Yang, L. Zhao, H. Dai and F. Xiao, "Mobility-Aware Computation Offloading in Satellite Edge Computing Networks," in *IEEE Transactions on Mobile Computing*, vol. 23, no. 10, pp. 9135–9149, Oct. 2024, doi: 10.1109/TMC.2024.3359759.
- [115] H. Chen, T. D. Todd, D. Zhao and G. Karakostas, "Task Class Partitioning for Mobile Computation Offloading," in *IEEE Internet of Things Journal*, vol. 11, no. 2, pp. 2534–2549, 15 Jan. 15, 2024, doi: 10.1109/JIOT.2023.3294887.
- [116] X. Wang, W. Xu and Z. Jin, "A Hidden Markov Model based dynamic scheduling approach for mobile cloud telemonitoring," 2017 IEEE EMBS International Conference on Biomedical & Health Informatics (BHI), Orlando, FL, USA, 2017, pp. 273–276, doi: 10.1109/BHI.2017.7897258.
- [117] Naik, B.B., Priyanka, B. & Ansari, M.S.A. Energy-efficient task offloading and efficient resource allocation for edge computing: a quantum inspired particle swarm optimization approach. *Cluster Comput* 28, 155 (2025). <https://doi-org.egateway.chennai.vit.ac.in/10.1007/s10586-024-04833-5>
- [118] Mansouri, W., Alohal, M.A., Alqahtani, H. et al. Deep convolutional neural network-based enhanced crowd density monitoring for intelligent urban planning on smart cities. *Sci Rep* 15, 5759 (2025). <https://doi-org.egateway.chennai.vit.ac.in/10.1038/s41598-025-90430-4>
- [119] Machidon, A.L., Pejović, V. Integrating control-theoretic predictive deep learning for resource-efficient computing systems. *J Intell Inf Syst* (2025). <https://doi-org.egateway.chennai.vit.ac.in/10.1007/s10844-025-00919-7>
- [120] Tian, H., Zhu, L. & Tan, L. A joint task caching and computation offloading scheme based on deep reinforcement learning. *Peer-to-Peer Netw. Appl.* 18, 57 (2025). <https://doi-org.egateway.chennai.vit.ac.in/10.1007/s12083-024-01836-2>
- [121] Shafaq, T., Mirza, T., Obidallah, W.J. et al. An energy-efficient task offloading in D2D-assisted IoT networks using matching algorithms. *J Supercomput* 81, 568 (2025). <https://doi-org.egateway.chennai.vit.ac.in/10.1007/s11227-025-06990-9>
- [122] Zheng, L., Tan, L. A decentralized scheme for multi-user edge computing task offloading based on dynamic pricing. *Peer-to-Peer Netw. Appl.* 18, 91 (2025). <https://doi-org.egateway.chennai.vit.ac.in/10.1007/s12083-025-01904-1>.
- [123] Kesavan V. T., R. V., Wong, W.K. et al. Reinforcement learning based Secure edge enabled multi-task scheduling model for internet of everything applications. *Sci Rep* 15, 6254 (2025). <https://doi-org.egateway.chennai.vit.ac.in/10.1038/s41598-025-89726-2>
- [124] TensorFlow. (n.d.). TensorFlow. <https://www.tensorflow.org/>
- [125] PyTorch. (n.d.). PyTorch. <https://pytorch.org/>
- [126] scikit-learn: machine learning in Python — scikit-learn 1.7.0 documentation. (n.d.). <https://scikit-learn.org/stable/>
- [127] CloudSim. (n.d.). CloudSim. Retrieved June 17, 2025, from <http://www.cloudbus.org/cloudsim/>
- [128] Cloudslab. (n.d.). GitHub - Cloudslab/iFogSim: The iFogSimToolkit (with its new release iFogSim2) for Modeling and Simulation of Resource Management Techniques in Internet of Things, Edge and Fog Computing Environments. In the new release Mobili Management, Microservice Management, and Dynamic Clustering mechanisms are added as new features. GitHub. <https://github.com/Cloudslab/iFogSim>
- [129] CagataySonmez. (n.d.). GitHub - CagataySonmez/EdgeCloudSIM: An environment for performance evaluation of edge computing systems. GitHub. <https://github.com/CagataySonmez/EdgeCloudSim>
- [130] Simulink - Simulation and Model-Based Design. (n.d.). <https://www.mathworks.com/products/simulink.html>
- [131] Open Ecosystem. (n.d.). Intel. <https://www.intel.com/content/www/us/en/developer/technology/open/overview.html>
- [132] Running Average Power Limit Energy Reporting CVE-2020-8694. (n.d.). Intel. <https://www.intel.com/content/www/us/en/developer/articles/technical/software-security-guidance/advisory-guidance/running-average-power-limit-energy-reporting.html>
- [133] Wireshark • undefined. (n.d.). Wireshark. <https://www.wireshark.org/>
- [134] pycryptodome. (2025, May 17). PyPI. <https://pypi.org/project/pycryptodome/>
- [135] Banerjee, U., Vashishtha, A., Saxena, M. (2010). Evaluation of the Capabilities of WireShark as a tool for Intrusion Detection. *International Journal of Computer Applications*, 6(7), 1–5. <https://doi.org/10.5120/1092-1427>
- [136] Dodiya, B., Singh, U. K. (2022). Malicious Traffic analysis using Wireshark by collection of Indicators of Compromise. *International Journal of Computer Applications*, 183(53), 1–6. <https://doi.org/10.5120/ijca2022921876>
- [137] Moffie, M., Boehm, O., Koyfman, A., Bin, E., Sztokman, E., Bhattacharjee, S., ... McGugan, J. (2025). Cryptoscope: Analyzing cryptographic usages in modern software. *arXiv preprint arXiv:2503.19531*.
- [138] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo and J. Zhang, "Edge Intelligence: Paving the Last Mile of Artificial Intelligence With Edge Computing," in *Proceedings of the IEEE*, vol. 107, no. 8, pp. 1738–1762, Aug. 2019, doi: 10.1109/JPROC.2019.2918951.
- [139] C. Zhang, P. Patras and H. Haddadi, "Deep Learning in Mobile and Wireless Networking: A Survey," in *IEEE Communications Surveys & Tutorials*, vol. 21, no. 3, pp. 2224–2287, thirdquarter 2019, doi: 10.1109/COMST.2019.2904897.

- [140] N. D. Lane et al., "DeepX: A Software Accelerator for Low-Power Deep Learning Inference on Mobile Devices," 2016 15th ACM/IEEE International Conference on Information Processing in Sensor Networks (IPSN), Vienna, Austria, 2016, pp. 1-12, doi: 10.1109/IPSN.2016.7460664.
- [141] Han, S., Mao, H., & Dally, W. J. (2016). Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding. International Conference on Learning Representations. arXiv:1510.00149.
- [142] Y. Wang, R. Zou, Y. Chen and Z. Gao, "Research on Pedestrian Detection Based on Jetson Xavier NX Platform and YOLOv4," 2023 4th International Symposium on Computer Engineering and Intelligent Communications (ISCEIC), Nanjing, China, 2023, pp. 373-377, doi: 10.1109/ISCEIC59030.2023.10271216.
- [143] J. V. Anchitaalagammai, S. Kavitha, R. Buurvidha, T. S. Santhiya, M. D. Roopa and S. S. Sankari, "Edge Artificial Intelligence for Real-Time Decision Making using NVIDIA Jetson Orin, Google Coral Edge TPU and 6G for Privacy and Scalability," 2025 International Conference on Visual Analytics and Data Visualization (ICVADV), Tirunelveli, India, 2025, pp. 150-155, doi: 10.1109/ICVADV63329.2025.10960953.
- [144] Choudhary, S., S, V., Bhavani, D. D., N, B., Tiwari, M., S, S. (2025). Edge AI Deploying artificial intelligence models on edge devices for Real-Time analytics. ITM Web of Conferences, 76, 01009. <https://doi.org/10.1051/itmconf/20257601009>
- [145] Banbury, C. R., Reddi, V. J., Lam, M., Fu, W., Fazel, A., Holleman, J., ... Yadav, P. (2020). Benchmarking tinyml systems: Challenges and direction. arXiv preprint arXiv:2003.04821.
- [146] R. Xu, F. Han and Q. Ta, "Deep Learning at Scale on NVIDIA V100 Accelerators," 2018 IEEE/ACM Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS), Dallas, TX, USA, 2018, pp. 23-32, doi: 10.1109/PMBS.2018.8641600.



ANANT KUMAR received the B.Tech degree from the Dr. M.G.R. Educational and Research Institute, Chennai, Tamilnadu, India. He is currently pursuing the Ph.D degree in the field of School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, Tamilnadu, India. His research interests include AI, ML, Computation Offloading, Cloud computing, and Internet of Things(IoT). He has published articles in reputed journals.



AMRIT PAL (Member, IEEE) received the B.Tech. degree from Kurukshetra University, Kurukshetra, India in 2011, the M.Tech. degree from National Institute of Technical Teachers' Training and Research, Bhopal, India in 2014, and the Ph.D. degree from Indian Institute of Information Technology Allahabad, India in 2020. He worked as an Assistant Professor at the Centre for Advanced Studies, AKTU, Lucknow, India. He is currently working as an Assistant Professor at the Vellore Institute of Technology, Chennai, India. His research interests include big data analytics, cloud computing, machine learning, and Internet of Things.

...